arXiv:1605.02406v2 [cs.RO] 10 Sep 2016

IJRR

A Random Finite Set Approach for Dynamic Occupancy Grid Maps with Real-Time Application

Dominik Nuss[1], Stephan Reuter[2], Markus Thom[2], Ting Yuan[1],
Gunther Krehl[1], Michael Maile[1], Axel Gern[1], Klaus Dietmayer[2]

***Abstract*—Grid mapping is a well established approach for environment perception in robotic and automotive applications. Early work suggests estimating the occupancy state of each grid cell in a robot's environment using a Bayesian filter to recursively combine new measurements with the current posterior state estimate of each grid cell. This filter is often referred to as binary Bayes filter (BBF). A basic assumption of classical occupancy grid maps is a stationary environment. Recent publications describe bottom-up approaches using particles to represent the dynamic state of a grid cell and outline prediction-update recursions in a heuristic manner. This paper defines the state of multiple grid cells as a random finite set, which allows to model the environment as a stochastic, dynamic system with multiple obstacles, observed by a stochastic measurement system. It motivates an original filter called the probability hypothesis density / multi-instance Bernoulli (PHD/MIB) filter in a top-down manner. The paper presents a real-time application serving as a fusion layer for laser and radar sensor data and describes in detail a highly efficient parallel particle filter implementation. A quantitative evaluation shows that parameters of the stochastic process model affect the filter results as theoretically expected and that appropriate process and observation models provide consistent state estimation results.**

I. Introduction

The beginning of grid mapping approaches took place in the field of robotics [1], [2]. A classic grid map divides the environment into single grid cells and estimates the occupancy probability for each cell. Since several measurements occur over time, the grid map combines these measurements with a Bayesian filter. A commonly used filter for this application is the binary Bayes filter, which combines measurements to estimate the binary state of a grid cell: free or occupied [3]. A restrictive assumption of the common binary Bayes filter application is that the environment is stationary. Furthermore, a common assumption of grid maps is the independence of individual grid cells which facilitates a fast implementation at the cost of approximation errors.

Today, grid maps are used in many automated vehicles [4], [5], [6]. Due to their explicit free-space estimation and their ability to represent arbitrarily shaped objects, grid maps are an important tool for collision avoidance. Moreover, the spatial grid structure provides a convenient fusion layer for data from different range finding sensors [7], [8]. In vehicle environment perception, the assumption of a stationary environment is obviously not fulfilled due to moving road users like vehicles or pedestrians.

Recently, several approaches have been presented to combine grid mapping and multi-object tracking. A well-known example is an approach called simultaneous localization, mapping and moving object tracking (SLAMMOT) [9], which retains a grid map and multiple object tracks at the same time and assigns object detections either to the grid map or to a tracked object. Other publications suggest associating grid cells directly to object tracks [10] or detecting object movement in multiple time frames of grid maps using a post-processing step [11]. Further approaches combine grid mapping and object tracking in a modular way [12], [13].

However, some of these approaches imply complicated environment perception architectures and are therefore not an appropriate choice for many applications. In 2006, Coué et al. proposed the Bayesian occupancy filter (BOF) [14] which uses a four-dimensional grid to estimate a two-dimensional environment. Here, two grid dimensions represent the spatial position and two grid dimensions represent the two-dimensional velocity of the obstacles. Thus, the BOF estimates object movement and explicitly considers it in its process model. The BOF motivated many applications [7], [15], but a problem is the high computational load caused by the large number of grid cells necessary to represent the environment appropriately.

An important improvement by Danescu et al. [16], [17] suggested to represent the dynamic state of a grid cell with particles resulting in a significant reduction of the computational load. In subsequent publications, Nègre et al. [18] and Tanzmeister et al. [19] independently proposed to represent only the dynamic part of a grid map with particles instead of all occupied grid cells. In 2015, Nuss et al. suggested to use the dynamic grid map as a fusion layer for laser and radar measurements [20], which would improve the overall performance of the dynamic grid map, especially the separation between moving and static obstacles.

In summary, previous work on dynamic grid maps based on particles shows promising results. Unfortunately, the proposed filters lack a stochastically rigorous definition of a multi-object state estimation problem. As such, they describe evolutionary algorithms (survival of the fittest) rather than Bayesian filters.

A. Contributions of this Paper

This paper models the dynamic state estimation of grid cells as a random finite set (RFS) problem. Finite set statistics

[1]Mercedes-Benz Research & Development North America, Inc., USA dominik.s.nuss@daimler.com or firstname.lastname@daimler.com, respectively

[2]Institute of Measurement, Control, and Microtechnology, Ulm University, Germany firstname.lastname@uni-ulm.de

S. Reuter is supported by the German Research Foundation (DFG) within the Transregional Collaborative Research Center SFB/TRR 62 "Companion-Technology for Cognitive Technical Systems".

(FISST) [21] provide a mathematical framework for the state estimation of multiple dynamic objects in a Bayesian sense. Well-known techniques from the field of FISST like the probability hypothesis density filter (PHD) [22] and the Bernoulli filter (BF) [23] are applied to estimate the dynamic state of grid cells. The resulting filter is called probability hypothesis density / multi-instance Bernoulli (PHD/MIB) filter. Modeling the estimation problem of a dynamic grid map in the random finite set domain yields substantial advantages. It gives every filter parameter a physical meaning and allows a generic and stochastically rigorous filter design for various estimation problems.

The key contributions of this paper are:

1) The definition of the dynamic state estimation of grid cells as an RFS problem and the derivation of the probability hypothesis density / multi-instance Bernoulli (PHD/MIB) filter, which takes into account the special form of measurement grids as they are common for grid mapping approaches.
2) The realization of the PHD/MIB filter with particles and an approximation in the Dempster-Shafer domain.
3) A detailed pseudo code description of a massively parallel, real-time capable approximation of the PHD/MIB filter.
4) Results of experiments with real-world data and evaluation of estimation error and consistency of the approximated PHD/MIB filter.

B. Paper Structure

The remainder of this paper is structured as follows. Section II gives an overview of published dynamic grid mapping approaches. Section III outlines mathematical basics of random finite set statistics. The PHD/MIB filter is derived in Sect. IV. A particle-based realization is presented in Sect. V and approximated in the Dempster-Shafer domain in Sect. VI. Section VII provides a detailed description of a highly-efficient parallel implementation, followed by the evaluation in Sect. VIII. Section IX presents the conclusion.

II. Dynamic Grid Mapping: An overview

This section provides an overview of current static and dynamic grid mapping approaches and discusses their advantages and drawbacks.

A. Static Grid Mapping

Classic occupancy grid maps divide the space into single grid cells and estimate the occupancy probability of each grid cell [1]–[3]. A binary grid cell state o_k at time k is considered either occupied or free: $o_k \in \{O, F\}$. The grid map updates the grid cell states when a new measurement arrives. For this purpose, an inverse sensor model assigns a discrete, binary occupancy probability $p_{z_{k+1}}(o_{k+1}|z_{k+1})$ individually to each grid cell based on the measurement z_{k+1} at time $k+1$. The result is called a measurement grid. To give a practical example, consider a laser range measurement consisting of several laser beams and the resulting measurement grid as depicted in Fig. 1. The inverse sensor model can be a heuristic model or the result of a machine learning process [2]. Usually, the position of the robot or vehicle in the grid map is estimated by a dead reckoning approach [3].

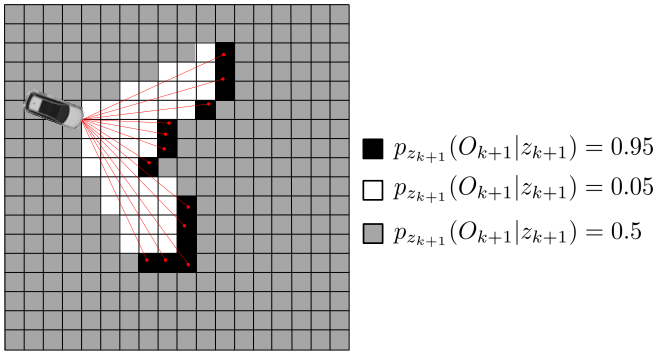


Fig. 1. Occupancy probabilities of two-dimensional grid cells, reasoning on a multi-beam laser range measurement. Grid cells with a high probability of being occupied are colored black, free grid cells are marked with white color. Grid cells with an unknown state (same probability for both occupied and free) are displayed in gray color.

The posterior occupancy probability $p_{o,k+1}(o_{k+1})$ at time $k+1$ results from the last posterior occupancy probability $p_{o,k}(o_k)$ at time k and the measurement-based estimate $p_{z_{k+1}}(o_{k+1}|z_{k+1})$ through [2]

$$p_{o,k+1}(o_{k+1}) = \frac{p_{z_{k+1}}(o_{k+1}|z_{k+1}) \cdot p_{o,k}(o_k)}{p_{z_{k+1}}(o_{k+1}|z_{k+1}) \cdot p_{o,k}(o_k) + p_{z_{k+1}}(\overline{o}_{k+1}|z_{k+1}) \cdot p_{o,k}(\overline{o}_k)}, \tag{1}$$

where $p(\overline{o}) = 1 - p(o)$ denotes the probability of the counter event of occupied or free, respectively.

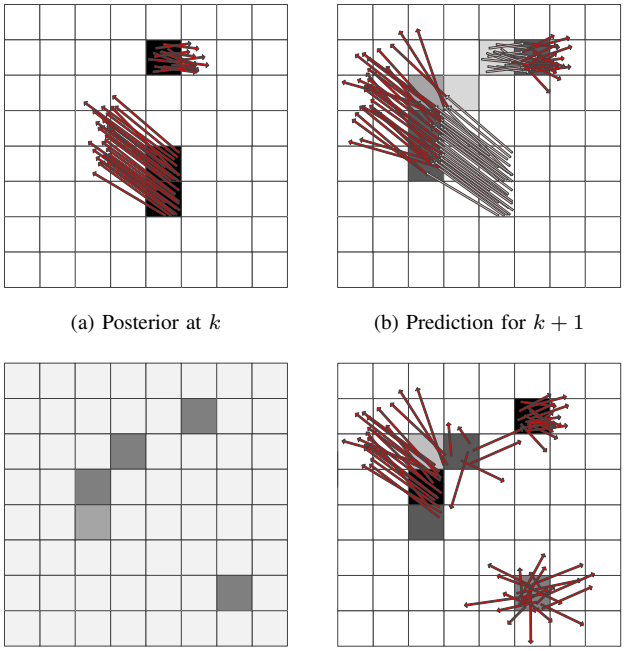
This is often referred to as binary Bayes filter due to the binary nature of the estimated state. Equation (1) holds if the prior probability for occupancy and free is equal, the measurements are independent of each other and the grid cell state does not change over time. An alternative approach is the forward sensor model, which estimates for each grid cell the occupancy likelihood function $g_{k+1}(z_{k+1}|o_{k+1})$ for the two feasible occupancy events $o_{k+1} \in \{O, F\}$. Then the update under the same assumptions as for (1) is given by

$$p_{o,k+1}(o_{k+1}) = \frac{g_{k+1}(z_{k+1}|o_{k+1}) \cdot p_{o,k}(o_k)}{g_{k+1}(z_{k+1}|o_{k+1}) \cdot p_{o,k}(o_k) + g_{k+1}(z_{k+1}|\overline{o}_{k+1}) \cdot p_{o,k}(\overline{o}_k)}. \tag{2}$$

Modeling likelihoods is more complicated than designing inverse sensor models and usually also computationally more expensive.

Equations (1) and (2) can be generalized to

$$p_{o,k+1}(o_{k+1}) = \frac{\alpha_{z_{k+1}} \cdot p_{o,k}(o_k)}{\alpha_{z_{k+1}} \cdot p_{o,k}(o_k) + p_{o,k}(\overline{o}_k)}, \tag{3}$$



(a) Posterior at k (b) Prediction for $k+1$

(c) Measurement grid at $k+1$ (d) Posterior at $k+1$

Fig. 2. Different states of dynamic grid map estimation recursion.

where $\alpha_{z_{k+1}}$ is the single measurement based occupancy probability ratio

$$\alpha_{z_{k+1}} = \frac{p_{z_{k+1}}(o_{k+1}|z_{k+1})}{p_{z_{k+1}}(\overline{o}_{k+1}|z_{k+1})}, \quad p_{z_{k+1}}(\overline{o}_{k+1}|z_{k+1}) > 0, \tag{4}$$

or the likelihood ratio

$$\alpha_{z_{k+1}} = \frac{g_{k+1}(z_{k+1}|o_{k+1})}{g_{k+1}(z_{k+1}|\overline{o}_{k+1})}, \quad g_{k+1}(z_{k+1}|\overline{o}_{k+1}) > 0, \tag{5}$$

respectively. As a conclusion, the binary Bayes filter requires either a likelihood ratio or a probability ratio for the update step. It will be shown later in Sect. IV-G that the binary Bayes filter (3) is a special case of the presented PHD/MIB filter, namely for the assumption of zero velocity in a deterministic process model.

B. Dynamic Grid Mapping

Since the assumption of a stationary environment is not realistic for typical traffic scenarios, several approaches to integrate object movement into grid maps have been proposed recently. This section compares four contributions from Danescu et al. [17], Tanzmeister et al. [19], Nègre et al. [18] and Nuss et al. [20].

All mentioned publications about particle-based dynamic grid maps estimate the occupancy probability and the dynamic state of grid cells in the vehicle environment. Further, all publications describe an algorithm consisting of a prediction and an update step as depicted in Fig. 2 and apply a resampling step to avoid degeneration.

1) State Representation: All mentioned publications represent the dynamic state of grid cells with particles, but the interpretation of a particle differs: [17] and [20] directly use the number of particles or the sum of particle weights in a grid cell as a measure for the occupancy probability of the grid cell. In contrast, [18] propagates an additional discrete probability distribution for the events free, static occupancy and dynamic occupancy for each grid cell. The particles then represent a velocity distribution for the dynamic case. The same events are used in [19] within a Dempster-Shafer framework [24]. To avoid aliasing problems, particles represent velocity and position of an occupancy in a grid cell in all mentioned publications, so the dynamic state of a grid cell is four-dimensional.

2) Prediction Step: All mentioned publications assume a process model with constant velocity and constant direction and propagate each single particle accordingly. All particles that are predicted into a certain grid cell represent the predicted dynamic state of the grid cell. However, the exact quantitative reasoning about the resulting predicted occupancy probability varies. Intuitively, the higher the number of particles or particle weights predicted into a grid cell, the higher is the predicted occupancy probability. An example is depicted in Fig. 2b.

3) Update Step: Updating the occupancy probability of a grid cell with a measurement grid is generally a binary Bayes problem and solved either by equation (1) or (2) or by equivalent update steps in the Dempster-Shafer framework [19], [20]. Due to the lack of a mathematically rigorous definition of a particle, all publications use different methods to normalize the particle weights in a grid cell after the update step to provide a consistent representation of the occupancy and the dynamic state of a grid map.

4) Resampling: All mentioned publications apply a resampling step to avoid degeneration. Similar to classic particle filters, the resampling step chooses to eliminate some particles and reproduce others instead, based on their weight. After the resampling step, all particles are assigned the same weight.

5) Initialization: If a measurement grid cell provides a high occupancy probability (or occupancy likelihood, in the forward case), but no particles were predicted into the corresponding grid cell, new particles must be initialized to represent the dynamic state of the grid cell. The initial distribution depends on the environment setting, but usually the velocity of movements is limited, e.g. by the maximum speed of a vehicle.

Neither [17] nor [19] describe the initialization step to any further detail than mentioned here. In a realistic scenario, a grid cell is not either empty or fully populated, but mostly something in between. Then the question arises how to divide the weight between predicted and initialized particles. Intuitively, the weight for newly initialized particles should rise with increasing measurement occupancy and decreasing predicted occupancy. Heuristic examples are provided by [18] and [20].

6) Occluded Areas: In practical applications, a grid map contains a high ratio of occluded and therefore unobserved grid cells. Populating unknown areas of the grid map with particles would result in a huge computational load. To avoid this, all mentioned publications only initialize particles in grid cells with a certain measured occupancy probability.

C. Discussion

The discussed particle-based BOFs show promising results. However, from a theoretical point of view many open questions remain. A prerequisite for Bayesian state estimation is the definition of a state space, a stochastic process describing the state transition and a stochastic observation process. All mentioned papers directly describe the propagation of particles without defining the estimation problem first. As a result, it is unclear what a particle represents. All mentioned publications explain that a particle represents a hypothesis for the dynamic state of an individual grid cell. However, during the prediction step, particles from various cells are predicted into another grid cell and jointly represent the state of the destination cell. The particles are not assigned to a specific object, instead they represent a hypothesis for the existence and state of a whole group of objects.

In other words, the particles jointly represent a set of occupied grid cells, where the number of occupied grid cells is a random process itself and must be estimated too. This cannot be explained with single-object Bayesian estimation theory. As a consequence, previous work cannot motivate prediction or update equations for a well-defined estimation problem. Especially the initialization of new particles remains unclear.

From a theoretical point of view, an environment containing a random but limited number of objects is a random finite set (RFS) [21]. The finite set statistics (FISST) are a mathematical framework providing a basis for Bayesian state estimation of multiple objects. The following section gives an introduction to the basics of FISST required for the derivation of dynamic grid mapping as an RFS estimation problem.

III. Random Finite Set Statistics

This section outlines the main concepts of finite set statistics and the multi-object Bayes filter. For further details, the reader is referred to [21] or [23].

A random finite set (RFS) is a finite set-valued random variable, i.e., a realization of an RFS consists of a random number of points or objects whose individual states are given by random vectors $x \in \mathbb{X}$ where $\mathbb{X}$ denotes the single-object state space. Thus, an RFS is represented by

$$\mathrm{X} = \{x^{(1)}, \dots, x^{(n)}\}$$

where $n \geq 0$ is a random variable and the special case $n = 0$ results in the empty set $\mathrm{X} = \emptyset$.

The cardinality distribution of an RFS is given by an arbitrary discrete distribution and the probability for an RFS representing exactly n objects is denoted by $\rho(n)$. For each cardinality $n > 0$, the RFS contains a set of probability density functions (PDFs)

$$\{f_n(x^{(1)}, \dots, x^{(n)}),\, n \in \mathbb{N} \mid \rho(n) > 0\},$$

i.e., the RFS supports several different state distributions for a single cardinality. Since an RFS is order independent, the multi-object probability density function (MPDF) is given by

$$\pi(\mathrm{X} = \{x^{(1)}, \dots, x^{(n)}\}) = \begin{cases} \rho(0) & \text{if } \mathrm{X} = \emptyset, \\ n! \cdot \rho(n) \cdot f_n(x^{(1)}, \dots, x^{(n)}) & \text{otherwise,} \end{cases} \quad (6)$$

where the factor $n!$ accounts for all possible permutations of the vectors $x^{(1)}, \dots, x^{(n)}$.

Since the number of objects is also a random variable, the set integral [21]

$$\int \pi(\mathrm{X})\delta\mathrm{X} = \pi(\emptyset) + \sum_{n=1}^{\infty} \frac{1}{n!} \int \pi(x^{(1)}, \dots, x^{(n)}) dx^{(1)} \cdots dx^{(n)} \quad (7)$$

has to be applied for the integration over an MPDF.

A. Multi-Object Bayes Filter

Conventional multi-object tracking is typically realized using several instances of a Kalman filter [25]. This provides an analytical solution to the single-object Bayes filter in case of Gaussian distributed states and measurements as well as linear motion and measurement models. The multi-object Bayes filter [21] is a generalization of the single-object Bayes filter which handles the uncertainty in the number of objects in a mathematically rigorous way.

If the multi-object density at time k is given by $\pi_k(\mathrm{X}_k)$, the predicted multi-object density is obtained by applying the Chapman-Kolmogorov equation:

$$\pi_+(\mathrm{X}_{k+1}) = \int f_+(\mathrm{X}_{k+1}|\mathrm{X}_k)\pi_k(\mathrm{X}_k)\delta\mathrm{X}_k. \quad (8)$$

Here, $f_+(\mathrm{X}_{k+1}|\mathrm{X}_k)$ denotes the multi-object transitional density which captures the appearance and disappearance of objects in addition to the movement of persisting objects. For a shorter notation, the index "+" expresses a prediction step from time k to time $k+1$, often noted as $k+1|k$.

The measurement update of the predicted multi-object density using a set of measurements Z_{k+1} is realized by applying Bayes' rule to yield

$$\pi_{k+1}(\mathrm{X}_{k+1}|\mathrm{Z}_{k+1}) = \frac{\gamma_{k+1}(\mathrm{Z}_{k+1}|\mathrm{X}_{k+1})\pi_+(\mathrm{X}_{k+1})}{\int \gamma_{k+1}(\mathrm{Z}_{k+1}|\mathrm{X}_{k+1})\pi_+(\mathrm{X}_{k+1})\delta\mathrm{X}_{k+1}}, \quad (9)$$

where the integral in the denominator is a set integral as defined in Eq. (7). Similar to the multi-object transitional density in the prediction step, the multi-object likelihood function $\gamma_{k+1}(\mathrm{Z}_{k+1}|\mathrm{X}_{k+1})$ has to incorporate the uncertainty of the measurement process, i.e., it has to model missed detections and false alarms.

A realization of the multi-object Bayes filter is possible using Sequential Monte Carlo (SMC) methods (e.g. [21], [26], [27]) or Generalized Labeled Multi-Bernoulli distributions [28], [29]. Further, several approximations like the Probability Hypothesis Density (PHD) filter [22], [23], [30], the Cardinalized PHD filter [31], [32], the Cardinality Balanced Multi-Bernoulli Filter [33] and the Labeled Multi-Bernoulli filter [34] have been proposed during the last decade.

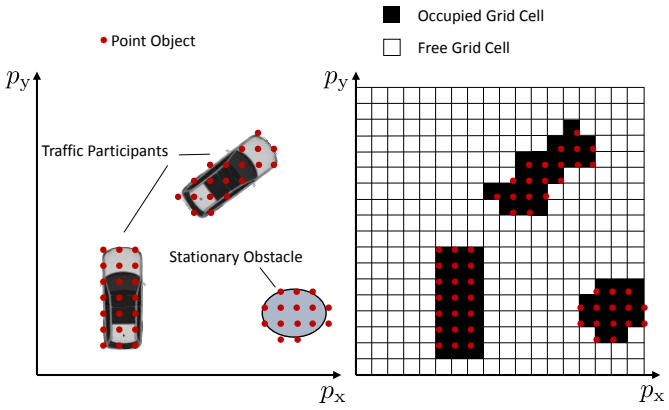


Fig. 3. Left: Real-world objects and corresponding point objects. Right: The number of point objects per real-world object equals the number of grid cells occupied by the real-world object.

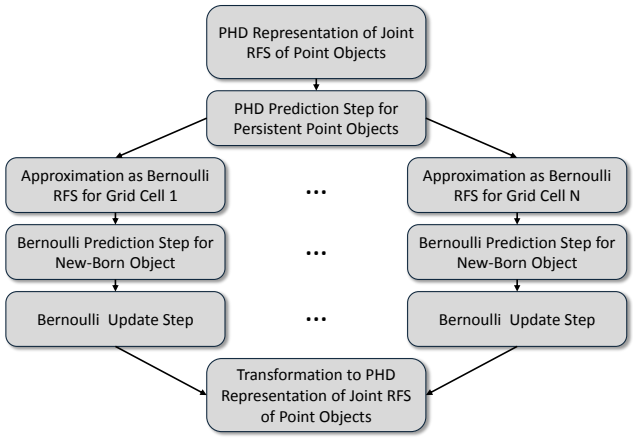


Fig. 4. Processing scheme of the PHD/MIB filter.

B. PHD and Bernoulli RFS

The PHD filter approximates the full multi-object density using the first statistical moment which is given by its intensity distribution or probability hypothesis density (PHD) [22]:

$$D(x) = E\left\{\sum_{w\in \mathrm{X}} \delta(x-w)\right\} = \int \sum_{w\in \mathrm{X}} \delta(x-w)\pi(\mathrm{X})\delta\mathrm{X}. \tag{10}$$

Here, $E\{\cdot\}$ denotes the expectation. Since $D(x)$ is an intensity distribution, the integral over $D(x)$ corresponds to the expected number of targets in this area.

An important multi-object distribution for the remainder of this contribution is the Bernoulli RFS. A Bernoulli RFS [21] is typically used to model scenarios where an object either exists with an existence probability r or does not exist with a probability of $1-r$. If the object exists, its spatial distribution is given by the single-object PDF $p(x)$. Consequently, the multi-object probability density follows

$$\pi(\mathrm{X}) = \begin{cases} 1-r & \text{if } \mathrm{X} = \emptyset, \\ r\cdot p(x) & \text{if } \mathrm{X} = \{x\}, \\ 0 & \text{if } |\mathrm{X}| \geq 2. \end{cases}$$

The intensity function or PHD of a Bernoulli RFS, which corresponds to the first statistical moment, is given by the product of the existence probability and the spatial distribution [21]:

$$D(x) = r\cdot p(x). \tag{11}$$

IV. The Probability Hypothesis Density / Multi-Instance Bernoulli (PHD/MIB) Filter

This section proposes to model the dynamic grid map using random finite set theory which facilitates to combine the Bernoulli filter and the PHD filter to recursively estimate the state of dynamic grid cells. These filters cover fundamental problems like object initialization or modeling of heterogeneous measurements in an elegant way. The result is a recursion called probability hypothesis density / multi-instance Bernoulli (PHD/MIB) filter. First, the environment model and the estimation problem are defined. Based hereon, the filter steps are outlined and the prediction and update equations of the PHD/MIB filter are derived.

A. Environment Definition and Filter Recursion Outline

The PHD/MIB filter estimates a random finite set consisting of so-called point objects. The relation between point objects and real-world objects depends on an underlying grid map and is shown in Fig. 3. A real-world object consists of at least one but possibly several point objects. The number of corresponding point objects per real-world object equals the number of grid cells occupied by the real-world object.

To provide an example for the state x of a point object, let x be a two-dimensional position and a two-dimensional velocity:

$$x = [p_\mathrm{x}\, p_\mathrm{y}\, v_\mathrm{x}\, v_\mathrm{y}]^\mathrm{T}. \tag{12}$$

However, the object state can be arbitrarily extended and can include additional attributes such as object height, color, semantic class, etc.

The goal of the PHD/MIB filter is to estimate the multi-object state of the point objects in a vehicle environment, which includes an estimate of the occupancy state of grid cells, as will be explained below. In the course of the filter recursion, the PID/MIB filter represents the random finite set of point objects in different forms. An overview of the filter recursion is depicted in Fig. 4. The posterior state is uniquely represented by its probability hypothesis density (PHD). The PHD/MIB filter prediction step simply applies the PHD filter prediction. In order to update the predicted RFS state with a measurement grid, the PHD/MIB filter approximates the point object state in each grid cell as a Bernoulli RFS and carries out the update step independently for each grid cell. Finally, the PHD/MIB filter transfers all instances of Bernoulli sets to a joint PHD to represent the posterior state. The individual filter steps are detailed below.

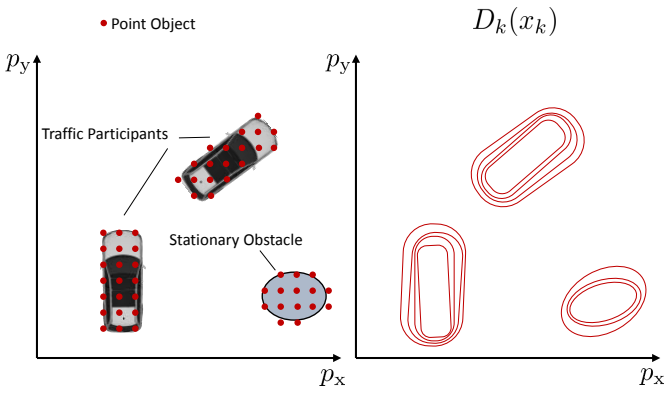


Fig. 5. Exemplary PHD of a traffic situation. Here, only the subspace of the two-dimensional position is visualized.

B. Multi-Object State Transition in PHD representation

The PHD/MIB filter expresses the posterior multi-object state of time step k by its PHD $D_k(x_k)$. In practical applications, PHDs are commonly represented by particles or Gaussian mixtures. However, the following derivation is independent of its practical representation form. Figure 5 shows an exemplary PHD of a traffic situation using contour lines of a Gaussian mixture. The birth process of point objects is defined by the birth PHD $\gamma_\mathrm{b}(x_{k+1})$ and the persistence probability of each point object is denoted by p_S. The standard prediction step of a PHD filter is then given by [22]

$$D_+(x_{k+1}) = \gamma_\mathrm{b}(x_{k+1}) + p_\mathrm{S} \int f_+(x_{k+1}|x_k) D_k(x_k) dx_k, \quad (13)$$

where $f_+(x_{k+1}|x_k)$ is the single-object transition density. An example for a process model which defines a transition density is the constant velocity (CV) model given by

$$x_{k+1} = \begin{pmatrix} 1 & 0 & T & 0 \\ 0 & 1 & 0 & T \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} x_k + \xi_k, \quad (14)$$

where ξ_k is the process noise and T is the time interval between k and $k+1$.

Since new-born objects will be handled in Bernoulli form, the prediction step of the PHD/MIB filter is only required to handle persisting objects and consequently simplifies to

$$D_{\mathrm{p},+}(x_{k+1}) = p_\mathrm{S} \int f_+(x_{k+1}|x_k) D_k(x_k) dx_k. \quad (15)$$

Here, the lower index p symbolizes the affiliation to persistent objects.

C. Approximation as Bernoulli Distributions

The PHD/MIB filter approximates the predicted PHD $D_{\mathrm{p},+}(x_{k+1})$ of persisting objects by multiple instances of independent Bernoulli RFSs, one for each grid cell. The interpretation is that each grid cell can either be occupied (a point object exists in the grid cell) of free (no point object exists in the grid cell). Each Bernoulli RFS instance models the possibility of object birth and the observation process including clutter. The birth process model is identical to the standard Bernoulli filter. The clutter process is different and will be derived for the special form of a measurement grid. The following part describes the Bernoulli filter steps for a single grid cell denoted by c.

The predicted Bernoulli RFS for a persistent point object in grid cell c is given by

$$\pi^{(c)}_{\mathrm{p},+}(\mathrm{X}_{k+1}) = \begin{cases} \overline{r}^{(c)}_{\mathrm{p},+} & \text{if } \mathrm{X}_{k+1} = \emptyset, \\ r^{(c)}_{\mathrm{p},+} \cdot p^{(c)}_{\mathrm{p},+}(x_{k+1}) & \text{if } \mathrm{X}_{k+1} = \{x_{k+1}\}, \end{cases} \quad (16)$$

where $r^{(c)}_{\mathrm{p},+}$ is the predicted existence probability of a persistent point object, $p^{(c)}_{\mathrm{p},+}(x_{k+1})$ is the predicted PDF of its state, and $\overline{r}^{(c)}_{\mathrm{p},+} = 1 - r^{(c)}_{\mathrm{p},+}$. The upper index (c) denotes values which are different for individual cells. From the definition of the PHD it follows that

$$r^{(c)}_{\mathrm{p},+} = \min \left(\int_{x_{k+1} \in c} D_{\mathrm{p},+}(x_{k+1}) dx_{k+1},\ 1 \right), \quad (17)$$

where the set $\{x_{k+1} | x_{k+1} \in c\}$ is the subset of $\mathbb{R}^4$ associated to grid cell c. Here, the limitation to a maximum value of 1 is a required approximation since the PHD prediction does not consider that each grid cell cannot be occupied by more than one point object. Further, the spatial distribution within the grid cell is given by

$$p^{(c)}_{\mathrm{p},+}(x_{k+1}) = \frac{D_{\mathrm{p},+}(x_{k+1})}{r^{(c)}_{\mathrm{p},+}} \text{ for all } x_{k+1} \in c \text{ and } r^{(c)}_{\mathrm{p},+} > 0. \quad (18)$$

D. Bernoulli RFS Birth Model

The standard Bernoulli filter defines the following birth model [23]: If no object exists at time step k, the existence probability of a new-born object at time step $k+1$ is given by the prior birth probability p_B. If an object exists at time step k, the existence probability of a new-born object at time $k+1$ is zero, independent of the survival of the old object. In the case of a new-born object, the single-object spatial birth PDF is given by $p_\mathrm{b}(x_{k+1})$.

The PHD/MIB filter assumes the same birth process as the standard Bernoulli filter. This results in the predicted existence probability $r^{(c)}_{\mathrm{b},+}$ of a new-born object with [1]

$$r^{(c)}_{\mathrm{b},+} = p_\mathrm{B} \cdot \left(1 - r^{(c)}_{\mathrm{p},+}\right). \quad (19)$$

[1]Formally, the probability of a birth event in time step $k+1$ depends on the existence probability of an object in time step k for the Bernoulli filter [23]. For simplicity, equation (19) neglects that fact and directly relates to the predicted existence probability for time step $k+1$. This is appropriate for process models with a high persistence probability ($p_\mathrm{S} \approx 1$).

Considering both cases of a persistent or new-born object leads to the predicted Bernoulli RFS

$$\pi_+^{(c)}(\mathrm{X}_{k+1}) = \begin{cases} 1 - r_{\mathrm{b},+}^{(c)} - r_{\mathrm{p},+}^{(c)} & \text{if } \mathrm{X}_{k+1} = \emptyset, \\ r_{\mathrm{b},+}^{(c)} \cdot p_{\mathrm{b}}(x_{k+1}) + r_{\mathrm{p},+}^{(c)} \cdot p_{\mathrm{p},+}^{(c)}(x_{k+1}) & \text{if } \mathrm{X}_{k+1} = \{x_{k+1}\}. \end{cases} \tag{20}$$

Note that the predicted Bernoulli RFS $\pi_+^{(c)}(\mathrm{X}_{k+1})$ represents both the predicted dynamic distribution and the predicted occupancy probability of the corresponding grid cell. The predicted occupancy probability $p_{o,+}^{(c)}(O_{k+1})$ equals the combined predicted existence probability $r_+^{(c)}$ of a persistent or a new-born point object in the grid cell, so by definition of the Bernoulli RFS it follows:

$$p_{o,+}^{(c)}(O_{k+1}) = r_+^{(c)} = r_{\mathrm{b},+}^{(c)} + r_{\mathrm{p},+}^{(c)}. \tag{21}$$

E. Bernoulli Observation Process

The standard Bernoulli filter expects Poisson distributed clutter detections [23]. Since in practical applications a measurement grid cell usually does not contain more than one measurement, this is not feasible.

The PHD/MIB defines the following Bernoulli RFS observation process instead: A measurement grid map provides an observation for each grid cell based on sensor data at time step $k+1$. The observation process of one grid cell is assumed independent of other grid cells.

In each grid cell either one measurement $z_{k+1}^{(c)}$ or no measurement occurs. The probability that a measurement occurs in the occupied grid cell c is the cell-specific and time-dependent true positive probability $p_{\mathrm{TP},k+1}^{(c)} \in (0,1)$. The probability that a measurement occurs in the empty cell c is the false positive probability $p_{\mathrm{FP},k+1}^{(c)} \in (0,1)$. The PDF of a false positive measurement is given by the clutter density $p_{\mathrm{cl}}(z)$.

A true positive measurement is associated to the point object in the grid cell with the association probability $p_{\mathrm{A},k+1}^{(c)}$. In this case its distribution is defined by the single-object likelihood function $g_{k+1}^{(c)}(z_{k+1}|x_{k+1})$. If the measurement is not associated to the point object, its PDF is also defined by the clutter density $p_{\mathrm{cl}}(z)$.

In practical terms, each measurement grid cell c contains the following data: the individual, time-dependent true positive and false positive probabilities $p_{\mathrm{TP},k+1}^{(c)}$ and $p_{\mathrm{FP},k+1}^{(c)}$. In case a measurement occurred in the cell it additionally provides the single-object likelihood function $g_{k+1}^{(c)}(z_{k+1}|x_{k+1})$ and the corresponding association probability $p_{\mathrm{A},k+1}^{(c)}$.

The resulting multi-object likelihood $\gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1}|\mathrm{X}_{k+1})$ for the measurement grid cell c calculates to

$$\begin{aligned} &\gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1} = \emptyset|\mathrm{X}_{k+1} = \emptyset) = \overline{p}_{\mathrm{FP},k+1}^{(c)}, \\ &\gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1} = \{z_{k+1}\}|\mathrm{X}_{k+1} = \emptyset) = p_{\mathrm{FP},k+1}^{(c)} \cdot p_{\mathrm{cl}}(z_{k+1}), \\ &\gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1} = \emptyset|\mathrm{X}_{k+1} = \{x_{k+1}\}) = \overline{p}_{\mathrm{TP},k+1}^{(c)}, \\ &\gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1} = \{z_{k+1}\}|\mathrm{X}_{k+1} = \{x_{k+1}\}) = \\ &\quad p_{\mathrm{TP},k+1}^{(c)} \underbrace{\left[p_{\mathrm{A},k+1}^{(c)} \cdot g_{k+1}^{(c)}(z_{k+1}|x_{k+1}) + \overline{p}_{\mathrm{A},k+1}^{(c)} \cdot p_{\mathrm{cl}}(z_{k+1})\right]}_{=: g_{\mathrm{A},k+1}^{(c)}(z_{k+1}|x_{k+1})}. \end{aligned} \tag{22}$$

F. Multi-Object Bayes Update Step

Ultimately, inserting the predicted Bernoulli RFS (20) and the multi-object likelihood (22) into the general multi-object update (9) leads to the posterior multi-object PDF $\pi_{k+1}^{(c)}(\mathrm{X}_{k+1}|\mathrm{Z}_{k+1})$:

$$\begin{aligned} &\pi_{k+1}^{(c)}(\mathrm{X}_{k+1} = \emptyset|\mathrm{Z}_{k+1} = \emptyset) = \frac{\overline{p}_{\mathrm{FP},k+1}^{(c)} \cdot \overline{r}_+^{(c)}}{\mu_{\{\emptyset\}}^{(c)}}, \\ &\pi_{k+1}^{(c)}(\mathrm{X}_{k+1} = \{x_{k+1}\}|\mathrm{Z}_{k+1} = \emptyset) = \frac{\overline{p}_{\mathrm{TP},k+1}^{(c)} \cdot \pi_+^{(c)}(\{x_{k+1}\})}{\mu_{\{\emptyset\}}^{(c)}}, \\ &\pi_{k+1}^{(c)}(\mathrm{X}_{k+1} = \emptyset|\mathrm{Z}_{k+1} = \{z_{k+1}\}) = \frac{p_{\mathrm{FP},k+1}^{(c)} \cdot p_{\mathrm{cl}}(z_{k+1}) \cdot \overline{r}_+^{(c)}}{\mu_{\{z\}}^{(c)}}, \\ &\pi_{k+1}^{(c)}(\mathrm{X}_{k+1} = \{x_{k+1}\}|\mathrm{Z}_{k+1} = \{z_{k+1}\}) = \\ &\quad \frac{p_{\mathrm{TP},k+1}^{(c)} \cdot g_{\mathrm{A},k+1}^{(c)}(z_{k+1}|x_{k+1}) \cdot \pi_+^{(c)}(\{x_{k+1}\})}{\mu_{\{z\}}^{(c)}}. \end{aligned} \tag{23}$$

Using (7), the normalization constants calculate to

$$\begin{aligned} \mu_{\{\emptyset\}}^{(c)} &= \int \gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1} = \emptyset|\mathrm{X}_{k+1}) \pi_+^{(c)}(\mathrm{X}_{k+1}) \delta\mathrm{X}_{k+1} \\ &= \overline{p}_{\mathrm{FP},k+1}^{(c)} \cdot \overline{r}_+^{(c)} + \overline{p}_{\mathrm{TP},k+1}^{(c)} \cdot r_+^{(c)} \end{aligned} \tag{24}$$

and

$$\begin{aligned} \mu_{\{z\}}^{(c)} &= \int \gamma_{k+1}^{(c)}(\mathrm{Z}_{k+1} = \{z_{k+1}\}|\mathrm{X}_{k+1}) \pi_+^{(c)}(\mathrm{X}_{k+1}) \delta\mathrm{X}_{k+1} \\ &= p_{\mathrm{FP},k+1}^{(c)} \cdot p_{\mathrm{cl}}(z_{k+1}) \cdot \overline{r}_+^{(c)} \\ &\quad + p_{\mathrm{TP},k+1}^{(c)} \int g_{\mathrm{A},k+1}^{(c)}(z_{k+1}|x_{k+1}) \cdot \pi_+^{(c)}(\{x_{k+1}\}) dx_{k+1}. \end{aligned} \tag{25}$$

The posterior $\pi_{k+1}^{(c)}(\mathrm{X}_{k+1})$ is a Bernoulli RFS and represents both the dynamic state and the occupancy probability of the corresponding grid cell. According to the definition of a Bernoulli RFS, the posterior occupancy probability of grid cell c is

$$p_{o,k+1}^{(c)}(O_{k+1}) = r_{k+1}^{(c)} = \int\limits_{x_{k+1} \in c} \pi_{k+1}^{(c)}(\{x_{k+1}\})\, dx_{k+1}. \tag{26}$$

After the update step, the PHD/MIB filter transforms all Bernoulli RFS instances into a joint PHD again. The joint PHD is simply the sum of all Bernoulli RFS instances:

$$D_{k+1}(x_{k+1}) = \sum_{c=1}^{C} \pi_{k+1}^{(c)}(\mathrm{X}_{k+1} = \{x_{k+1}\}), \tag{27}$$

where c denotes the index of the corresponding grid cell of each Bernoulli RFS instance and C is the total number of grid cells. This closes the recursion.

G. Relation between PHD/MIB filter and binary Bayes filter

The proposed PHD/MIB filter is a generalization of the binary Bayes filter which does not rely on the assumption of a static environment. Consequently, the filter equations should simplify to the well-known equations of the binary Bayes filter for a static process model.

Proposition: Assume a deterministic, static process model in the PHD/MIB filter, so that the predicted intensity distribution for time step $k+1$ is equivalent to the posterior distribution at time k, i.e.

$$D_+(x_{k+1}) = D_k(x_k) \;\forall\; x_{k+1} = x_k \in \mathbb{X}. \tag{28}$$

Further, assume the measurement likelihood $g^{(c)}_{k+1}(z_{k+1}|x_{k+1})$ and the clutter density $p_{\text{cl}}(z_{k+1})$ are equal uniform distributions in a limited subset $\mathbb{Z}_\text{s}$ of the measurement space, i.e.

$$g^{(c)}_{k+1}(z_{k+1}|x_{k+1}) = p_{\text{cl}}(z_{k+1}) = \begin{cases} \theta & \text{if } z_{k+1} \in \mathbb{Z}_\text{s}, \\ 0 & \text{otherwise}, \end{cases} \tag{29}$$

where $\theta > 0$ is constant. Then the propagation of the posterior occupancy probability $r^{(c)}_k = p^{(c)}_{o,k}(o_k = O_k)$ at time k to the posterior occupancy probability $r^{(c)}_{k+1} = p^{(c)}_{o,k+1}(o_{k+1} = O_{k+1})$ at time $k+1$ of the PHD/MIB filter reduces to the generalized binary Bayes update (3)

$$p^{(c)}_{o,k+1}(O_{k+1}) = \frac{\alpha^{(c)}_{z_{k+1}}(O_{k+1}) \cdot p^{(c)}_{o,k}(O_k)}{\alpha^{(c)}_{z_{k+1}}(O_{k+1}) \cdot p^{(c)}_{o,k}(O_k) + p^{(c)}_{o,k}(F_{k+1})}, \tag{30}$$

with

$$\alpha^{(c)}_{z_{k+1}}(O_{k+1}) = \begin{cases} \frac{\overline{p}^{(c)}_{\text{TP},k+1}}{\overline{p}^{(c)}_{\text{FP},k+1}} \text{ if } \mathrm{Z}_{k+1} = \emptyset, \\ \frac{p^{(c)}_{\text{TP},k+1}}{p^{(c)}_{\text{FP},k+1}} \text{ if } \mathrm{Z}_{k+1} = \{z_{k+1}\}. \end{cases} \tag{31}$$

Recall that O and F are the two possible cases occupied and free, respectively of the occupancy state o.

Proof: Due to the static process model (28), the Bernoulli distribution for each grid cell does not change during the prediction step:

$$\pi^{(c)}_+ = \pi^{(c)}_k. \tag{32}$$

In case of no measurement in grid cell c, i.e. $\mathrm{Z}_{k+1} = \emptyset$, the posterior existence probability of an oject in grid cell c is given by

$$\begin{aligned} r^{(c)}_{k+1} &\overset{(26)}{=} \int_{x_{k+1} \in c} \pi^{(c)}_{k+1}(\{x_{k+1}\}|\emptyset)\, dx_{k+1} \\ &\overset{(23),(32)}{=} \int_{x_{k+1} \in c} \frac{1}{\mu^{(c)}_{\{\emptyset\}}} \cdot \overline{p}^{(c)}_{\text{TP},k+1} \cdot \pi^{(c)}_k(\{x_{k+1}\})\, dx_{k+1} \\ &\overset{(24),(32)}{=} \frac{\overline{p}^{(c)}_{\text{TP},k+1} \cdot r^{(c)}_k}{\overline{p}^{(c)}_{\text{FP},k+1} \cdot \overline{r}^{(c)}_k + \overline{p}^{(c)}_{\text{TP},k+1} \cdot r^{(c)}_k} \end{aligned} \tag{33}$$

and for $\mathrm{Z}_{k+1} = \{z_{k+1}\}$ it follows

$$\begin{aligned} r^{(c)}_{k+1} &\overset{(26)}{=} \int_{x_{k+1} \in c} \pi^{(c)}_{k+1}(\{x_{k+1}\}|\{z_{k+1}\})\, dx_{k+1} \\ &\overset{(23),(32)}{=} \int_{x_{k+1} \in c} \frac{p^{(c)}_{\text{TP},k+1}}{\mu^{(c)}_{\{z\}}} \cdot g^{(c)}_{\text{A},k+1} \cdot \pi^{(c)}_k(\{x_{k+1}\})\, dx_{k+1} \\ &\overset{(25),(29),(32)}{=} \frac{p^{(c)}_{\text{TP},k+1} \cdot g^{(c)}_{\text{A},k+1} \cdot r^{(c)}_k}{p^{(c)}_{\text{FP},k+1} \cdot p_{\text{cl}} \cdot \overline{r}^{(c)}_k + p^{(c)}_{\text{TP},k+1} \cdot g^{(c)}_{\text{A},k+1} \cdot r^{(c)}_k} \\ &\overset{(29)}{=} \frac{p^{(c)}_{\text{TP},k+1} \cdot r^{(c)}_k}{p^{(c)}_{\text{FP},k+1} \cdot \overline{r}^{(c)}_k + p^{(c)}_{\text{TP},k+1} \cdot r^{(c)}_k}. \square \end{aligned} \tag{34}$$

V. Particle Realization of the PHD/MIB Filter

The PHD/MIB filter can be realized in different ways. A main characteristic of the realization is the representation form of the state PHD. This section describes the particle realization of the PHD/MIB filter.

A. Posterior Representation

Particles are random samples of the posterior PHD at time step k. A particle set consists of ν particles and their weights $\{x^{(i)}_k, w^{(i)}_k\}_{i=1}^{\nu}$. Together they approximate the posterior as

$$D_k(x_k) \approx \sum_{i=1}^{\nu} w^{(i)}_k \delta(x_k - x^{(i)}_k). \tag{35}$$

Here, δ is the Dirac delta function which satisfies

$$\int f(x)\delta(x)dx = f(0). \tag{36}$$

B. Prediction of Persistent Objects

To represent predicted persistent objects, the prediction draws particles by sampling the proposal density q_{k+1}:

$$x^{(i)}_{\text{p},+} \sim q_{k+1}(\cdot|x^{(i)}_k, \mathrm{Z}_{k+1}). \tag{37}$$

The index p depicts that these particles represent a persistent point object. The remaining part of the section assumes it is possible to sample from the transition density f_+, which is used as proposal density:

$$q_{k+1}(x_{k+1}|x^{(i)}_k, \mathrm{Z}_{k+1}) = f_+(x_{k+1}|x^{(i)}_k). \tag{38}$$

The sampling provides the predicted particle set $\{x^{(i)}_{\text{p},+}, w^{(i)}_{\text{p},+}\}_{i=1}^{\nu}$ for time step $k+1$, where the particle weights are multiplied with the persistence probability:

$$w^{(i)}_{\text{p},+} = p_\text{S} \cdot w^{(i)}_k. \tag{39}$$

The set represents the predicted PHD of persistent point objects:

$$D_{\text{p},+}(x_{k+1}) \approx \sum_{i=1}^{\nu} w^{(i)}_{\text{p},+} \delta(x_{k+1} - x^{(i)}_{\text{p},+}). \tag{40}$$

C. Transition from a PHD to Multiple Instances of Bernoulli RFSs

As depicted in Fig. 4, the PHD/MIB filter now transforms the representation form from the joint PHD to individual, independent Bernoulli RFSs for each grid cell. Accordingly, the following steps are carried out for each grid cell c individually.

Let

$$\{x_{\mathrm{p},+}^{(i,c)}, w_{\mathrm{p},+}^{(i,c)}\}_{i=1}^{\nu_{\mathrm{p},+}^{(c)}} \tag{41}$$

be the set of particles predicted into grid cell c at time step $k+1$. The symbol $\nu_{\mathrm{p},+}^{(c)}$ represents the number of particles predicted into grid cell c at time step $k+1$.

To keep the notation simple, consider the set (41) as already truncated, i.e., the sum of weights does not exceed 1. If the sum of predicted particle weights in one grid cell exceeds 1, the weights must be normalized to sum up to a number smaller than 1.

According to (17), the sum of predicted particle weights in grid cell c then gives the predicted existence probability $r_{\mathrm{p},+}^{(c)}$ of a persistent object in cell c:

$$r_{\mathrm{p},+}^{(c)} = \sum_{i=1}^{\nu_{\mathrm{p},+}^{(c)}} w_{\mathrm{p},+}^{(i,c)} \ \in [0,1]. \tag{42}$$

D. Prediction of New-Born Objects

According to (20), the predicted existence probability $r_{\mathrm{b},+}^{(c)}$ of a new-born object in grid cell c is given by

$$r_{\mathrm{b},+}^{(c)} = p_{\mathrm{B}} \cdot \left(1 - r_{\mathrm{p},+}^{(c)}\right). \tag{43}$$

Predicted new-born objects in grid cell c are represented by the particle set

$$\{x_{\mathrm{b},+}^{(i,c)}, w_{\mathrm{b},+}^{(i,c)}\}_{i=1}^{\nu_{\mathrm{b},+}^{(c)}}. \tag{44}$$

The particles of this set are sampled from the birth distribution:

$$x_{\mathrm{b},+}^{(i,c)} \sim p_{\mathrm{b}}(\cdot). \tag{45}$$

The number of new-born particles $\nu_{\mathrm{b},+}^{(c)}$ for each grid cell c is a design parameter of the system. It should be chosen individually for each grid cell, depending on the probability of a birth event.

Since the new-born particle weights sum up to the predicted existence probability $r_{\mathrm{b},+}^{(c)}$ (19) of a new-born object in grid cell c, the weight of each new-born particle is given by

$$w_{\mathrm{b},+}^{(i,c)} = \frac{r_{\mathrm{b},+}^{(c)}}{\nu_{\mathrm{b},+}^{(c)}}. \tag{46}$$

E. Predicted Bernoulli RFS

Together, the persistent and the new-born particle sets represent the predicted Bernoulli RFS $\pi_{+}^{(c)}(\mathrm{X}_{k+1})$ in grid cell c:

$$\begin{aligned}\pi_{+}^{(c)}(\{x_{k+1}\}) \approx &\sum_{i=1}^{\nu_{\mathrm{p},+}^{(c)}} w_{\mathrm{p},+}^{(i,c)} \delta(x_{k+1} - x_{\mathrm{p},+}^{(i,c)}) \\ &+ \sum_{i=1}^{\nu_{\mathrm{b},+}^{(c)}} w_{\mathrm{b},+}^{(i,c)} \delta(x_{k+1} - x_{\mathrm{b},+}^{(i,c)})\end{aligned} \tag{47}$$

and

$$\pi_{+}^{(c)}(\emptyset) = 1 - r_{\mathrm{p},+}^{(c)} - r_{\mathrm{b},+}^{(c)}. \tag{48}$$

F. Particle Update

Assume a measurement grid map taken at time step $k+1$ provides for each grid cell an observation as stated above in Sect. IV. The update step adapts the weights of the particle set (47).

The update rules for persistent and new-born particles are identical. The notation system uses the weight symbol w_* with $* \in \{\mathrm{p},\mathrm{b}\}$ in equations that are identical for both persistent particle weights w_{p} and new-born particle weights w_{b}.

In case a measurement occurred in measurement grid cell c, unnormalized adapted particle weights $\widetilde{w}_{*,k+1}^{(i,c)}$ are calculated according to (23):

$$\widetilde{w}_{*,k+1}^{(i,c)} = p_{\mathrm{TP},k+1}^{(c)} \cdot g_{\mathrm{A},k+1}^{(c)}(z_{k+1}|x_{*,+}^{(i,c)}) \cdot w_{*,+}^{(i,c)}. \tag{49}$$

The normalized weights are given by

$$w_{*,k+1}^{(i,c)} = \frac{\widetilde{w}_{*,k+1}^{(i,c)}}{\mu_{\{z\}}^{(c)}} \tag{50}$$

with (25)

$$\begin{aligned}\mu_{\{z\}}^{(c)} \approx &\, p_{\mathrm{FP},k+1}^{(c)} \cdot p_{\mathrm{cl}}(z_{k+1}) \cdot \overline{r}_{+}^{(c)} \\ &+ \sum_{i=1}^{\nu_{\mathrm{p},+}^{(c)}} \widetilde{w}_{\mathrm{p},k+1}^{(i,c)} + \sum_{i=1}^{\nu_{\mathrm{b},+}^{(c)}} \widetilde{w}_{\mathrm{b},k+1}^{(i,c)}.\end{aligned} \tag{51}$$

In case no measurement occurred in measurement grid cell c, the update rule for both the persistent and new-born particles to calculate adapted weights $w_{*,k+1}^{(i,c)}$ is according to (23):

$$w_{*,k+1}^{(i,c)} = \frac{\overline{p}_{\mathrm{TP},k+1}^{(c)}}{\overline{p}_{\mathrm{TP},k+1}^{(c)} \cdot r_{+}^{(c)} + \overline{p}_{\mathrm{FP},k+1}^{(c)} \cdot \overline{r}_{+}^{(c)}} w_{*,+}^{(i,c)}. \tag{52}$$

Notice that for multi-object distributions, normalization does not mean all particle weights sum up to 1. Instead, the sum of updated particle weights equals the posterior existence probability $r_{k+1}^{(c)}$ of a point object in grid cell c at time step $k+1$, which is also the posterior occupancy probability $p_{o,k+1}^{(c)}(O_{k+1})$ of the grid cell:

$$r_{k+1}^{(c)} = p_{o,k+1}^{(c)}(O_{k+1}) = \sum_{i=1}^{\nu_{\mathrm{p},k+1}^{(c)}} w_{\mathrm{p},k+1}^{(i,c)} + \sum_{i=1}^{\nu_{\mathrm{b},k+1}^{(c)}} w_{\mathrm{b},k+1}^{(i,c)}. \tag{53}$$

The posterior Bernoulli RFS $\pi_{k+1}^{(c)}(\mathrm{X}_{k+1})$ of grid cell c is given by:

$$\pi_{k+1}^{(c)}(\{x_{k+1}\}) \approx \sum_{i=1}^{\nu_{\mathrm{p},k+1}^{(c)}} w_{\mathrm{p},k+1}^{(i,c)} \delta(x_{k+1} - x_{\mathrm{p},k+1}^{(i,c)}) + \sum_{i=1}^{\nu_{\mathrm{b},k+1}^{(c)}} w_{\mathrm{b},k+1}^{(i,c)} \delta(x_{k+1} - x_{\mathrm{b},k+1}^{(i,c)}) \quad (54)$$

and

$$\pi_{k+1}^{(c)}(\emptyset) = \overline{r}_{k+1}^{(c)} = 1 - r_{k+1}^{(c)}. \quad (55)$$

G. Joint PHD Representation

The PHD/MIB represents the posterior multi-object state of all point objects in the environment by its PHD. The transformation from multiple instances of Bernoulli RFSs to a joint PHD is given by (27)

$$D_{k+1}(x_{k+1}) \approx \sum_{i \in [1, \nu_{\mathrm{p},k+1}^{(c)}], c \in [1,C]} w_{\mathrm{p},k+1}^{(i,c)} \delta(x_{k+1} - x_{\mathrm{p},k+1}^{(i,c)}) + \sum_{i \in [1, \nu_{\mathrm{b},k+1}^{(c)}], c \in [1,C]} w_{\mathrm{b},k+1}^{(i,c)} \delta(x_{k+1} - x_{\mathrm{b},k+1}^{(i,c)}). \quad (56)$$

Usually, particle filter realizations of a PHD filter provide only the persistent part of the posterior PHD as output [23]. Depending on the application, new-born particles considerably increase the uncertainty of the estimated state of objects. So it is often beneficial to consider their influence on the state estimation only after another recursion.

H. Resampling

For many applications it is important to keep the overall number of used particles constant. Therefore, the PHD/MIB filter resamples the constant number of ν particles from the joint posterior particle set. For each particle, the probability to be drawn is proportional to its weight. Let $\{x_{k+1}^{(i)}, w_{k+1}^{(i)}\}_{i=1}^{\nu}$ be the set of resampled particles and their weights. The new weights of the particles are all equal and normalized to sum up to the same value as the posterior weights of the persistent and the new-born particles together:

$$\sum_{i \in [1,\nu]} w_{k+1}^{(i)} = \sum_{i \in [1, \nu_{\mathrm{p},k+1}^{(c)}], c \in [1,C]} w_{\mathrm{p},k+1}^{(i,c)} + \sum_{i \in [1, \nu_{\mathrm{b},k+1}^{(c)}], c \in [1,C]} w_{\mathrm{b},k+1}^{(i,c)}. \quad (57)$$

VI. Real-time Approximation with Dempster-Shafer Theory of Evidence

For some application scenarios, the presented particle realization of the PHD/MIB filter might not be real-time capable. A possible reason are huge unobserved areas in grid maps. Since the presented particle realization of the PHD/MIB filter represents potential point objects in unobserved areas with particles, it requires a large number of them. All mentioned publications of particle-based dynamic grid maps [17]–[20] use particles only for occupied grid cells, not for unobserved grid cells. One possibility to distinguish between unobserved and occupied cells is to use Dempster-Shafer masses of evidence [24], [35], [36] instead of occupancy probabilities. Both [19] and [20] create particles only in areas with evidence for occupancy.

This section presents a coarse approximation of the particle-based PHD/MIB filter, applying the Dempster-Shafer theory of evidence. The resulting approximation will be referred to as DS-PHD/MIB filter. The DS-PHD/MIB filter is able to run with a substantially reduced number of particles compared to the original PHD/MIB filter and is also easier to implement. An efficient, massively parallelized implementation of the DS-PHD/MIB filter will be presented in Sect. VII.

A. State Representation

An introduction to the Dempster-Shafer theory of evidence and grid maps can be found in [4], [37], [38]. The DS-PHD/MIB filter represents the occupancy state of a grid cell with a basic belief assignment (BBA) $m : 2^{\Omega} \rightarrow [0, 1]$. The frame of discernment Ω contains the events occupied and free: $\Omega = \{O, F\}$. So each grid cell stores a mass for occupied $m(O)$ and a mass for free $m(F)$. The propagation of these masses over time are carried out separately. The propagation of the mass for free is estimated as in a static grid map. The propagation of the mass for occupied is motivated by the PHD/MIB filter.

The DS-PHD/MIB filter represents the posterior state of an individual grid cell c at time k with the particle set $\{x_k^{(i,c)}, w_k^{(i,c)}\}_{i=1}^{\nu_k^{(c)}}$ and the mass for free $m_k^{(c)}(F_k)$. Here, the sum of particle weights represents the mass for occupied:

$$m_k^{(c)}(O_k) = \sum_{i=1}^{\nu_k^{(c)}} w_k^{(i,c)}. \quad (58)$$

The occupancy probability $p_{o,k}^{(c)}(O)$ in a grid cell is given by the pignistic transformation

$$p_{o,k}^{(c)}(O_k) = m_k^{(c)}(O_k) + 0.5 \cdot (1 - m_k^{(c)}(O_k) - m_k^{(c)}(F_k)). \quad (59)$$

The distribution of the particles approximates the spatial PDF $p_k^{(c)}(x_k)$ of a point object in grid cell c:

$$p_k^{(c)}(x_k) \approx \frac{1}{m_k^{(c)}(O_k)} \sum_{i=1}^{\nu_k^{(c)}} w_k^{(i,c)} \delta(x_k - x_k^{(i,c)}). \quad (60)$$

B. Prediction

The DS-PHD/MIB filter applies (14) and (39) to predict particles to the next time step. In analogy to the PHD/MIB filter (41), let $\{x_{\mathrm{p},+}^{(i,c)}, w_{\mathrm{p},+}^{(i,c)}\}_{i=1}^{\nu_{\mathrm{p},+}^{(c)}}$ be the set of particles predicted into cell c at time step $k+1$. Again, the predicted weights are truncated, so the sum of predicted weights in one grid cell is limited to 1.

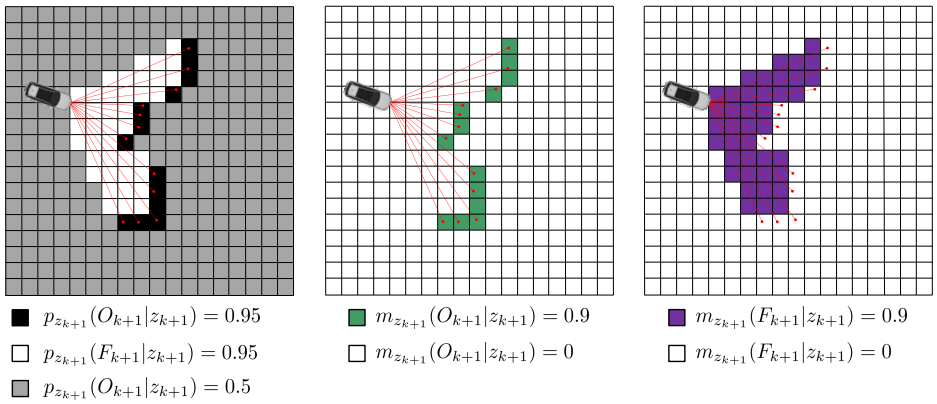


Fig. 6. A simple occupancy measurement grid originated by a laser measurement with multiple beams. The red lines represent laser beams that hit obstacles at their ends. The grid on the left-hand side provides an occupancy probability $p_{z_{k+1}}(o_{k+1}|z_{k+1})$ for each grid cell. The middle figure and the figure on the right-hand side show the same occupancy measurement grid represented by evidences for occupied $m_{z_{k+1}}(O_{k+1}|z_{k+1})$ (middle) and free $m_{z_{k+1}}(F_{k+1}|z_{k+1})$ (right), respectively. The pignistic transformation of the Dempster-Shafer grid results in the classical grid on the left-hand side [20]. The DS-PHD/MIB filter initializes new particles only in green grid cells.

The DS-PHD/MIB filter estimates the predicted occupancy mass of grid cell c by

$$m_{\mathrm{p},+}^{(c)}(O_{k+1}) = \sum_{i=1}^{\nu_{\mathrm{p},+}^{(c)}} w_{\mathrm{p},+}^{(i,c)}. \tag{61}$$

The predicted mass for free is modeled as in a static grid map and given by

$$m_{\mathrm{p},+}^{(c)}(F_{k+1}) = \min\left[\alpha(T) m_k^{(c)}(F_k)\,,\; 1 - m_{\mathrm{p},+}^{(c)}(O_{k+1})\right], \tag{62}$$

where the discount factor $\alpha(T) \in [0,1]$ models the decreasing prediction reliability, depending on the time interval T between two update steps. Since the sum of masses cannot exceed 1, the predicted free space evidence is limited accordingly.

C. Update

The PHD/MIB filter considers both the existence probability and the spatial distribution of point objects in a joint Bayesian innovation step, formally derived as a Bernoulli filter. The DS-PHD/MIB filter does not formally derive the update step, but uses heuristically designed, simplified update equations with the goal of modeling the probabilistic update equations of the PHD/MIB filter as close as possible in the Dempster-Shafer domain.

The DS-PHD/MIB approximation updates the existence probability of a point object in grid cell c independently of its spatial distribution. Accordingly, the DS-PHD/MIB filter expects the following information in each measurement grid cell:

- The observed occupancy BBA $m_{z_{k+1}}^{(c)} : 2^{\{O,F\}} \to [0,1]$,
- the spatial likelihood function $g_{k+1}^{(c)}(z_{k+1}|x_{k+1})$, and
- the association probability $p_{\mathrm{A},k+1}^{(c)}$ between the likelihood function $g_{k+1}^{(c)}(z_{k+1}|x_{k+1})$ and the point object.

Figure 6 shows an example for a measurement grid with occupancy BBAs. An example for a likelihood function $g_{k+1}^{(c)}(z_{k+1}|x_{k+1})$ and the association probability $p_{\mathrm{A},k+1}^{(c)}$ in measurement grid cells can be found in [20], where it results from radar doppler measurements.

1) Existence Update: The DS-PHD/MIB filter approximates the existence update by simply combining the predicted BBF $m_{\mathrm{p},+}^{(c)}$ and the observed BBA $m_{z_{k+1}}^{(c)}$ of the corresponding measurement grid cell with the Dempster-Shafer rule of combination (see [4]):

$$m_{k+1}^{(c)} = m_{\mathrm{p},+}^{(c)} \oplus m_{z_{k+1}}^{(c)}. \tag{63}$$

2) Birth Model: The DS-PHD/MIB filter splits the mass for occupied into two parts: occupied by a persistent object and occupied by a new-born object, denoted as:

$$m_{k+1}^{(c)}(O_{k+1}) = \varrho_{\mathrm{p},k+1}^{(c)} + \varrho_{\mathrm{b},k+1}^{(c)}. \tag{64}$$

Assume the PHD/MIB filter updates the state of a point object with a uniformly distributed likelihood $g_{k+1}^{(c)}(z_{k+1}|x_{k+1})$. Then the relation between the updated existence probability $r_{\mathrm{b},k+1}^{(c)}$ of a new-born object and the updated existence probability $r_{\mathrm{p},k+1}^{(c)}$ of a persistent object results in

$$\frac{r_{\mathrm{b},k+1}^{(c)}}{r_{\mathrm{p},k+1}^{(c)}} = \frac{r_{\mathrm{b},+}^{(c)}}{r_{\mathrm{p},+}^{(c)}} = \frac{p_{\mathrm{B}}\left[1 - r_{\mathrm{p},+}^{(c)}\right]}{r_{\mathrm{p},+}^{(c)}}. \tag{65}$$

Analogously, the DS-PHD/MIB models the relation between masses for a new-born and a persistent object as

$$\frac{\varrho_{\mathrm{b},k+1}^{(c)}}{\varrho_{\mathrm{p},k+1}^{(c)}} = \frac{p_{\mathrm{B}}\left[1 - \varrho_{\mathrm{p},k+1}^{(c)}\right]}{\varrho_{\mathrm{p},k+1}^{(c)}}. \tag{66}$$

Combining (64) and (66) delivers the resulting masses for a new-born and a persistent object:

$$\varrho_{\mathrm{b},k+1}^{(c)} = \frac{m_{k+1}^{(c)}(O_{k+1}) \cdot p_{\mathrm{B}}\left[1 - m_{\mathrm{p},+}^{(c)}(O_{k+1})\right]}{m_{\mathrm{p},+}^{(c)}(O_{k+1}) + p_{\mathrm{B}}\left[1 - m_{\mathrm{p},+}^{(c)}(O_{k+1})\right]}, \tag{67}$$

$$\varrho_{\mathrm{p},k+1}^{(c)} = m_{k+1}^{(c)}(O_{k+1}) - \varrho_{\mathrm{b},k+1}^{(c)}. \tag{68}$$

3) Spatial Update: The DS-PHD/MIB filter provides three particle sets to approximate the posterior spatial distribution $p^{(c)}_{k+1}(x_{k+1})$. The first particle set represents a persistent object and results from the set predicted into grid cell c, denoted as $\{x^{(i,c)}_{\mathrm{p},+}, w^{(i,c)}_{\mathrm{p},+}\}_{i=1}^{\nu^{(c)}_{\mathrm{p},+}}$. It is updated by multiplying the weights with the spatial measurement likelihood $g^{(c)}_{k+1}(z_{k+1}|x_{k+1})$. This leads to the unnormalized updated weights

$$\widetilde{w}^{(i,c)}_{\mathrm{p},k+1} = g^{(c)}_{k+1}(z_{k+1}|x^{(i,c)}_{\mathrm{p},+}) \cdot w^{(i,c)}_{\mathrm{p},+}. \tag{69}$$

The particle states remain unchanged:

$$x^{(i,c)}_{\mathrm{p},k+1} = x^{(i,c)}_{\mathrm{p},+}. \tag{70}$$

The normalized particle weights are given by

$$w^{(i,c)}_{\mathrm{p},k+1} = p^{(c)}_{\mathrm{A},k+1} \cdot \mu^{(c)}_{\mathrm{A}} \cdot \widetilde{w}^{(i,c)}_{\mathrm{p},k+1} + \left(1 - p^{(c)}_{\mathrm{A},k+1}\right) \cdot \mu^{(c)}_{\overline{\mathrm{A}}} \cdot w^{(i,c)}_{\mathrm{p},+}, \tag{71}$$

with

$$\mu^{(c)}_{\mathrm{A}} = \left[\sum_{i=1}^{\nu^{(c)}_{\mathrm{p},+}} \widetilde{w}^{(i,c)}_{\mathrm{p},k+1}\right]^{-1} \cdot \varrho^{(c)}_{\mathrm{p},k+1} \tag{72}$$

and

$$\mu^{(c)}_{\overline{\mathrm{A}}} = \left[\sum_{i=1}^{\nu^{(c)}_{\mathrm{p},+}} w^{(i,c)}_{\mathrm{p},+}\right]^{-1} \cdot \varrho^{(c)}_{\mathrm{p},k+1} = \frac{\varrho^{(c)}_{\mathrm{p},k+1}}{m^{(c)}_{\mathrm{p},+}(O_{k+1})}. \tag{73}$$

Equation (71) considers that with a probability of $(1-p^{(c)}_{\mathrm{A},k+1})$, the likelihood function $g^{(c)}_{k+1}(z_{k+1}|x^{(i,c)}_{\mathrm{p},+})$ is not associated with the point object in the grid cell. In this case, the weight update and normalization step serves solely to normalize the predicted particle weights in such a way that they sum up to the posterior persistent occupancy pass $\varrho^{(c)}_{\mathrm{p},k+1}$.

The second and third particle sets represent new-born objects. For computational efficiency reasons, they are only created in grid cells where the corresponding measurement grid cell reports a mass for occupied: $m^{(c)}_{z_{k+1}}(O_{k+1}) > 0$. The second particle set $\{x^{(i,c)}_{\mathrm{A},k+1}, w^{(i,c)}_{\mathrm{A},k+1}\}_{i=1}^{\nu^{(c)}_{\mathrm{A},k+1}}$ represents a new-born object under the assumption that the spatial measurement $z^{(c)}_{k+1}$ in grid cell c is associated to the point object in grid cell c. The particles are sampled from the probability density function $p^{(c)}_{x_{k+1}}(x_{k+1}|z^{(c)}_{k+1})$ of the state x_{k+1} given the measurement $z^{(c)}_{k+1}$ in grid cell c:

$$x^{(i,c)}_{\mathrm{A},k+1} \sim p^{(c)}_{x_{k+1}}(\cdot|z^{(c)}_{k+1}). \tag{74}$$

The weight of each particle in the second set can directly be calculated to

$$w^{(i,c)}_{\mathrm{A},k+1} = \frac{p^{(c)}_{\mathrm{A},k+1} \cdot \varrho^{(c)}_{\mathrm{b},k+1}}{\nu^{(c)}_{\mathrm{A},k+1}}. \tag{75}$$

Details about creating a probability density function $p^{(c)}_{x_{k+1}}(x_{k+1}|z^{(c)}_{k+1})$ of the state x_{k+1} given the measurement $z^{(c)}_{k+1}$ can be found in [23], p. 38.

The third particle set $\{x^{(i,c)}_{\overline{\mathrm{A}},k+1}, w^{(i,c)}_{\overline{\mathrm{A}},k+1}\}_{i=1}^{\nu^{(c)}_{\overline{\mathrm{A}},k+1}}$ represents a new-born object under the assumption that the spatial measurement $z^{(c)}_{k+1}$ in grid cell c is not associated to the point object in grid cell c. The particles of this set are sampled from the birth distribution $b_{k+1}(x_{k+1})$:

$$x^{(i,c)}_{\overline{\mathrm{A}},k+1} \sim b_{k+1}(\cdot) \tag{76}$$

The weight of each particle in the third set can directly be calculated to

$$w^{(i,c)}_{\overline{\mathrm{A}},k+1} = \frac{\left(1 - p^{(c)}_{\mathrm{A},k+1}\right) \cdot \varrho^{(c)}_{\mathrm{b},k+1}}{\nu^{(c)}_{\overline{\mathrm{A}},k+1}}. \tag{77}$$

When creating the second and the third particle set, the individual particle numbers $\nu^{(c)}_{\mathrm{A},k+1}$ and $\nu^{(c)}_{\overline{\mathrm{A}},k+1}$ of each grid cell should relate to their corresponding occupancy masses.

Finally, the posterior spatial state distribution of the point object in grid cell c at time $k+1$ is given by

$$p^{(c)}_{k+1}(x_{k+1}) \approx \frac{1}{m^{(c)}_{k+1}(O_{k+1})} \sum_{i=1}^{\nu^{(c)}_{k+1}} w^{(i,c)}_{k+1} \delta(x_{k+1} - x^{(i,c)}_{k+1}), \tag{78}$$

where the set $\{x^{(i,c)}_{k+1}, w^{(i,c)}_{k+1}\}_{i=1}^{\nu^{(c)}_{k+1}}$ is the union of all three particle sets created in the spatial update step. The grid cell additionally stores the posterior mass for free $m^{(c)}_{k+1}(F_{k+1})$ as calculated in (63), which completes the posterior state together with the particle set.

D. Resampling

The resampling step is identical to the resampling step of the original PHD/MIB filter.

VII. Parallel Implementation

This section describes an implementation of the particle-based DS-PHD/MIB filter with a focus on massively parallel processing systems such as graphics processing units.

A. Parallelization Challenges

Particles can naturally be processed in parallel, but here a challenge is to assign each particle to its corresponding grid cell in an efficient way. The assignment is necessary to predict the grid cell occupancy mass (61) and to calculate the normalization factor during the update step (71). Another challenge is to calculate statistical moments of grid cells as mean and variance of the velocity in a balanced way: the calculation time should be independent of the number of particles assigned to a grid cell.

The proposed solution sorts the particles after the prediction step according to the grid cell index they have been predicted into. Sorting particles has a time complexity quasilinear in the number of particles. Although the parallelization potential of sorting is somewhat limited, there are sophisticated sorting algorithms capable of achieving a high throughput especially on massively parallel architectures [39]. The availability of

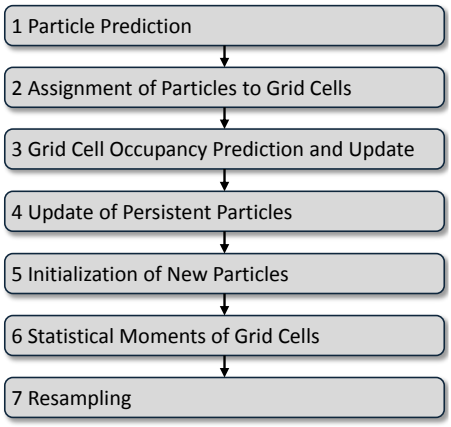


Fig. 7. Overview of the PHD/MIB implementation, broken down into seven steps.

a sorted particle array has several advantages: First, the assignment of sorted particles to grid cells is straightforward as will be detailed below. Furthermore, particle state values can be efficiently accumulated in separate arrays, similar to an integral image data structure [40]. This facilitates calculation of a grid cell's statistical moments with a computational complexity independent of the number of particles assigned to the cell.

Another advantage of sorting particles is a simple overall implementation since all remaining advanced problems can then be solved with standard routines. Highly efficient parallel implementations of these routines are available for graphics processing units, where for the parallel implementation of the particle-based DS-PHD/MIB filter sampling of random numbers [41], sorting [42] and accumulation have been employed.

B. Implementation Details

In the following, implementation details of the proposed parallel algorithm are described. The auxiliary data structures rendering the algorithm particularly efficient are given as follows. All particles and grid cells are stored in the *particle_array* and *grid_cell_array* arrays, respectively. Assume a measurement grid map with the same dimensions as the grid map is already available and measurement grid cells are stored in the array *meas_cell_array*. Details about efficient measurement grid calculation can be found in [43]. For modeling noise processes, a sufficient amount of random numbers is sampled beforehand during idle times and stored in extra arrays. The parallel DS-PHD/MIB recursion is summarized in Fig. 7 and outlined in the following sections.

1) Particle Prediction: Algorithm 1 provides pseudo code for the particle prediction. The algorithm predicts all particles in parallel applying equations (14) and (39). This includes calculating the new grid cell index of each particle after prediction. The appropriate amount of random numbers should be sampled in a separate step in advance, so the prediction step just needs to look them up.

2) Assignment of Particles to Grid Cells: Pseudo code for the particle assignment is given in Algorithm 2. First, the algorithm sorts all particles according to the grid cell index they have been predicted into. Each grid cell can store two particle indices. They represent the first and last index of the particle group that has been predicted into the grid cell. For the assignment, each particle checks if it is the first or the last particle of a group with the same index. If so, it writes its index into the according grid cell. Since there can only be at most one first or last particle per grid cell, the assignment can run in parallel without any writing conflicts.

3) Grid Cell Occupancy Prediction and Update: Algorithm 3 details the occupancy update. The goal of this step is to calculate for each grid cell the predicted and updated occupancy BBA. First, the algorithm accumulates in parallel all particle weights and stores the result in the array *weight_array_accum*. The remaining part of the algorithm is carried out in parallel for all grid cells. Each cell reads two values from *weight_array_accum*. The first value is the accumulated particle weight of all preceding grid cells excluding its own weight, the second value is the accumulated particle weight of all preceding grid cells including its own weight. The cell simply subtracts the first value from the second value to calculate its predicted occupancy mass according to (61) with constant time complexity. The predicted free mass is calculated according to (62).

Each cell reads the observed occupancy BBA from the corresponding measurement grid cell and combines it with its predicted occupancy BBA according to (63) to calculate its updated occupancy BBA. This enables the grid cell to separate the posterior occupancy mass $m_{k+1}^{(c)}(O_{k+1})$ into the part $\varrho_{\mathrm{p},k+1}^{(c)}$ for a persistent object and the part $\varrho_{\mathrm{b},k+1}^{(c)}$ for a new-born object. Each cell stores the part $\varrho_{\mathrm{b},k+1}^{(c)}$ for a new-born object in a separate array, which will be used later to calculate the number of newly drawn particles for this cell.

4) Update of Persistent Particles: The update of persistent particles is described in Algorithm 4. Each particle has stored its corresponding grid cell index during the prediction step, which is assumed the same as the corresponding measurement grid cell index. All persistent particles calculate in parallel their unnormalized updated weight according to (69). These weights are then accumulated in the array *weight_array_accum*. Recall that each grid cell has already stored the index range of its corresponding particles in Algorithm 2. Consequently, in a parallel **for** loop, each grid cell can look up the accumulated weight of its updated, unnormalized particles in *weight_array_accum* analogously to Algorithm 3. Each grid cell uses the result to calculate its normalization component $\mu_{\mathrm{A}}^{(c)}$ (72) and stores it. The other normalization component $\mu_{\overline{\mathrm{A}}}^{(c)}$ as given by (73) can directly be calculated and stored in the grid cell. In the next parallel **for** loop over all particles, each particle uses the grid map as a lookup table for its normalization components $\mu_{\mathrm{A}}^{(c)}$ and $\mu_{\overline{\mathrm{A}}}^{(c)}$ and normalizes itself (71).

5) Initialization of New Particles: Algorithm 5 depicts pseudo code for the particle initialization. New particles are stored in the array *birth_particle_array*. The total number of new particles for all grid cells ν_{b} remains constant over time, which is feasible for many real-time applications. The goal of

Algorithm 1 Particle Prediction

particle_array ▷ This array stores particles including weight and corresponding grid cell index (constant size ν)
grid_cell_array ▷ This array stores the grid cells (constant size C)
p_S ▷ Persistence probability of point objects, is a design parameter of the process model
for $i \in \{0, \ldots, \text{length}(\textit{particle_array}) - 1\}$ **do** ▷ Parallel for loop over all particles
    predict(*particle_array*, i, *p_S*) ▷ Applies (14) and (39), calculates new grid cell index and stores it inside the particle

Algorithm 2 Assignment of Particles to Grid Cells

weight_array ▷ This array will be an additional storage for the particle weights (constant size ν)
sort(*particle_array*) ▷ Sorts by the grid cell index the particle has been predicted into
for $i \in \{0, \ldots, \text{length}(\textit{particle_array}) - 1\}$ **do** ▷ Parallel for loop over all particles
    $j \leftarrow$ get_grid_cell_idx(*particle_array*[i]) ▷ Reads the grid cell index of the predicted particle
    if is_first_particle(*particle_array*, i) **then** ▷ Checks if i is the first particle of a group with same grid cell index
        set_particle_start_idx(*grid_cell_array*, j, i) ▷ Sets i as particle start index of grid cell j in *grid_cell_array*
    if is_last_particle(*particle_array*, i) **then** ▷ Checks if i is the last particle of a group with same grid cell index
        set_particle_end_index(*grid_cell_array*, j, i) ▷ Sets i as particle end index of grid cell j in *grid_cell_array*
    weight_array[i] $\leftarrow$ get_particle_weight(*particle_array*[i]) ▷ Copies weight of particle i to *weight_array* at index i

Algorithm 3 Grid Cell Occupancy Prediction and Update

meas_cell_array ▷ This array stores the measurement grid cells (constant size C)
born_masses_array ▷ This array will store the mass for a new-born object per grid cell (constant size C)
p_B ▷ Birth probability of point objects, is a design parameter of the process model
weight_array_accum $\leftarrow$ accumulate(*weight_array*) ▷ Inclusively accumulates all particle weights to *weight_array_accum*
for $j \in \{0, \ldots, \text{length}(\textit{grid_cell_array}) - 1\}$ **do** ▷ Parallel for loop over all grid cells
    start_idx $\leftarrow$ get_particle_start_idx(*grid_cell_array*[j]) ▷ Gets start index in *particle_array* of cell j
    end_idx $\leftarrow$ get_particle_end_idx(*grid_cell_array*[j]) ▷ Gets end index in *particle_array* of cell j
    m_occ_pred $\leftarrow$ subtract(*weight_array_accum*, *start_idx*, *end_idx*) ▷ Calculates predicted occupancy mass of cell j (61)
    m_free_pred $\leftarrow$ predict_free_mass(*grid_cell_array*[j]) ▷ Predicts free mass of cell j (62)
    m_occ_up $\leftarrow$ update_o(*m_occ_pred*, *m_free_pred*, *meas_cell_array*[j]) ▷ Combination to posterior occ. mass (63)
    m_free_up $\leftarrow$ update_f(*m_occ_pred*, *m_free_pred*, *meas_cell_array*[j]) ▷ Combination to posterior free mass (63)
    rho_b $\leftarrow$ separate_newborn_part(*m_occ_pred*, *m_occ_up*, *p_B*) ▷ Calculate new-born part of posterior occupancy mass (67)
    rho_p $\leftarrow$ *m_occ_up* $-$ *rho_b* ▷ Calculates remaining persistent part of posterior occupancy mass (68)
    born_masses_array[j] $\leftarrow$ *rho_b* ▷ Stores new-born part of posterior occupancy mass of cell j in *born_masses_array*
    store_values(*rho_b*, *rho_p*, *m_free_up*, *grid_cell_array*, j) ▷ Stores updated BBA in grid cell j

Algorithm 4 Update of Persistent Particles

for $i \in \{0, \ldots, \text{length}(\textit{particle_array}) - 1\}$ **do** ▷ Parallel for loop over all persistent particles
    weight_array[i] $\leftarrow$ update_unnorm(*particle_array*, i, *meas_cell_array*) ▷ Calculates unnormalized weight update (69)
weight_array_accum $\leftarrow$ accumulate(*weight_array*) ▷ Accumulates unnormalized weights of persistent particles
for $j \in \{0, \ldots, \text{length}(\textit{grid_cell_array}) - 1\}$ **do** ▷ Parallel for loop over all grid cells
    start_idx $\leftarrow$ get_particle_start_idx(*grid_cell_array*[j]) ▷ Gets start index in *particle_array* of cell j
    end_idx $\leftarrow$ get_particle_end_idx(*grid_cell_array*[j]) ▷ Gets end index in *particle_array* of cell j
    m_occ_accum $\leftarrow$ subtract(*weight_array_accum*, *start_idx*, *end_idx*) ▷ Calculate accumulated unnormalized updated particle weight of cell j
    rho_p $\leftarrow$ get_pers_occ_mass(*grid_cell_array*[j]) ▷ Gets persistent part of posterior occupancy mass of cell j
    mu_A $\leftarrow$ calc_norm_assoc(*m_occ_accum*, *rho_p*) ▷ Calculates normalization component for the case of an associated measurement (72)
    mu_UA $\leftarrow$ calc_norm_unassoc(*grid_cell_array*[j]) ▷ Calculates normalization component for the case of an unassociated measurement (73)
    set_normalization_components(*grid_cell_array*, j, *mu_A*, *mu_UA*) ▷ Stores *mu_A* and *mu_UA* as normalization components in grid cell j
for $i \in \{0, \ldots, \text{length}(\textit{particle_array}) - 1\}$ **do** ▷ Parallel for loop over all persistent particles
    weight_array[i] $\leftarrow$ normalize(*particle_array*[i], *grid_cell_array*) ▷ Normalizes particle weights (71)

Algorithm 5 Initialization of New Particles

birth_particle_array ▷ This array will be the storage for new-born particles for this recursion (constant size ν_b)
particle_orders_array_accum $\leftarrow$ accumulate(*born_masses_array*) ▷ Accumulates mass part of new-born object of each cell
normalize_particle_orders(*particle_orders_array_accum*, ν_b) ▷ Normalizes the particle orders to a total number of ν_b
for $j \in \{0, \ldots, \text{length}(\textit{grid_cell_array}) - 1\}$ **do** ▷ Parallel for loop over all grid cells
    start_idx $\leftarrow$ calc_start_idx(*particle_orders_array_accum*, j) ▷ Calculates first index in *birth_particle_array* of cell j
    end_idx $\leftarrow$ calc_end_idx(*particle_orders_array_accum*, j) ▷ Calculates last index in *birth_particle_array* of cell j
    num_new_particles $\leftarrow$ *end_idx* $-$ *start_idx* $+ 1$ ▷ Stores number of new particles for cell j
    p_A $\leftarrow$ get_assoc_probability(*meas_cell_array*[j]) ▷ Reads association probability to spatial measurement of cell j
    nu_A $\leftarrow$ calc_num_assoc(*num_new_particles*, *p_A*) ▷ Calculates number of new associated particles (79) and (80)
    nu_UA $\leftarrow$ *num_new_particles* $-$ *nu_A* ▷ Calculates number of new unassociated particles
    w_A $\leftarrow$ calc_weight_assoc(*nu_A*,*p_A*, *born_masses_array*[j]) ▷ Calculates weight of an associated new particle (75)
    w_UA $\leftarrow$ calc_weight_unassoc(*nu_UA*,*p_A*, *born_masses_array*[j]) ▷ Calculates weight of an unassociated new particle (77)
    store_weights(*w_A*, *w_UA*, *grid_cell_array*, j) ▷ Stores weights for new particles in cell j
    for $i \in \{\textit{start_idx} \ldots \textit{start_idx} + \textit{nu_A}\}$ **do** ▷ For loop over new associated particles of cell j
        set_cell_idx_A(*birth_particle_array*, i, j) ▷ Sets j as grid cell index of new particle i with flag for associated
    for $i \in \{\textit{start_idx} + \textit{nu_A} + 1 \ldots \textit{end_idx}\}$ **do** ▷ For loop over new unassociated particles of cell j
        set_cell_idx_UA(*birth_particle_array*, i, j) ▷ Sets j as grid cell index of new particle i with flag for unassociated
for $i \in \{0, \ldots, \text{length}(\textit{birth_particle_array}) - 1\}$ **do** ▷ Parallel for loop over all new-born particles
    initialize_new_particle(*birth_particle_array*, i, *grid_cell_array*) ▷ Initializes new-born particle i according to (74) or (76)

Algorithm 6 Statistical Moments of Grid Cells

vel_x_array ▷ Separate storage for the velocity in x-direction of particles (constant size ν)
vel_y_array ▷ Separate storage for the velocity in y-direction of particles (constant size ν)
vel_x_squared_array ▷ Separate storage for the squared velocity in x-direction of particles (constant size ν)
vel_y_squared_array ▷ Separate storage for the squared velocity in y-direction of particles (constant size ν)
vel_xy_array ▷ Separate storage for the multiplied velocities in both directions of particles (constant size ν)
for $i \in \{0, \dots, \text{length}(\textit{particle_array}) - 1\}$ **do** ▷ Parallel for loop over all persistent particles
    $w \leftarrow \textit{weight_array}[i]$ ▷ Stores the updated, normalized weight of the persistent particle i
    vel_x $\leftarrow$ get_vel_x(*particle_array*$[i]$) ▷ Stores x-velocity of particle i
    vel_y $\leftarrow$ get_vel_y(*particle_array*$[i]$) ▷ Stores y-velocity of particle i
    vel_x_array$[i] \leftarrow w *$ *vel_x* ▷ Stores weighted x-velocity of particle i
    vel_y_array$[i] \leftarrow w *$ *vel_y* ▷ Stores weighted y-velocity of particle i
    vel_x_squared_array$[i] \leftarrow w *$ *vel_x* $*$ *vel_x* ▷ Stores weighted squared x-velocity of particle i
    vel_y_squared_array$[i] \leftarrow w *$ *vel_y* $*$ *vel_y* ▷ Stores weighted squared y-velocity of particle i
    vel_xy_array$[i] \leftarrow w *$ *vel_x* $*$ *vel_y* ▷ Stores weighted product of x- and y-velocity of particle i
vel_x_array_accum $\leftarrow$ accumulate(*vel_x_array*) ▷ Accumulates velocities in x-direction
vel_y_array_accum $\leftarrow$ accumulate(*vel_y_array*) ▷ Accumulates velocities in y-direction
vel_x_squared_array_accum $\leftarrow$ accumulate(*vel_x_squared_array*) ▷ Accumulates squared velocities in x-direction
vel_y_squared_array_accum $\leftarrow$ accumulate(*vel_y_squared_array*) ▷ Accumulates squared velocities in y-direction
vel_xy_array_accum $\leftarrow$ accumulate(*vel_xy_array*) ▷ Accumulates product of velocities in x- and y-direction
for $j \in \{0, \dots, \text{length}(\textit{grid_cell_array}) - 1\}$ **do** ▷ Parallel for loop over all grid cells
    rho_p $\leftarrow$ get_pers_occ_mass(*grid_cell_array*, j) ▷ Gets persistent part of posterior occupancy mass in grid cell j
    start_idx $\leftarrow$ get_particle_start_idx(*grid_cell_array*$[j]$) ▷ Gets start index in *particle_array* of cell j
    end_idx $\leftarrow$ get_particle_end_idx(*grid_cell_array*$[j]$) ▷ Gets end index in *particle_array* of cell j
    mean_x_vel $\leftarrow$ calc_mean(*vel_x_array_accum*, *start_idx*, *end_idx*, *rho_p*) ▷ Applies (81)
    mean_y_vel $\leftarrow$ calc_mean(*vel_y_array_accum*, *start_idx*, *end_idx*, *rho_p*) ▷ Applies (81)
    var_x_vel $\leftarrow$ calc_variance(*vel_x_squared_array_accum*, *start_idx*, *end_idx*, *rho_p*, *mean_x_vel*) ▷ Applies (83)
    var_y_vel $\leftarrow$ calc_variance(*vel_y_squared_array_accum*, *start_idx*, *end_idx*, *rho_p*, *mean_y_vel*) ▷ Applies (83)
    covar_xy_vel $\leftarrow$ calc_covariance(*vel_xy_array_accum*, *start_idx*, *end_idx*, *rho_p*, *mean_x_vel*, *mean_y_vel*) ▷ Applies (84)
    store(*grid_cell_array*, j, *mean_x_vel*, *mean_y_vel*, *var_x_vel*, *var_y_vel*, *covar_xy_vel*) ▷ Store stochastic moments in grid cell j

Algorithm 7 Resampling

rand_array ▷ Array with sorted, equally distributed random numbers (constant size ν)
idx_array_resampled ▷ Array with indices of resampled particles (constant size ν)
particle_array_next ▷ Particle array for the next time step (constant size ν)
joint_weight_array_accum $\leftarrow$ accumulate(*weight_array*, *birth_weight_array*) ▷ Accumulates normalized particle weights
idx_array_resampled $\leftarrow$ calc_resampled_indeces(*joint_weight_array_accum*, *rand_array*) ▷ Calculates resampled particle indices
for $i \in \{0, \dots, \text{length}(\textit{particle_array}) - 1\}$ **do** ▷ Parallel for loop over all persistent particles of the next time step
    particle_array_next$[i]$ $\leftarrow$ copy_particle(*particle_array*, *birth_particle_array*, *idx_array_resampled*$[i]$) ▷ Copy resampled particle

this algorithm is to initialize for each grid cell a certain number of new particles, which is proportional to the new-born part of its updated occupancy mass $\varrho^{(c)}_{\mathrm{b},k+1}$. Therefore, the algorithm accumulates new-born occupancy masses of all grid cells in the array *particle_orders_array*. Then it normalizes each value of the array to sum up to the discrete number ν_{b}. The normalized array then serves as a lookup table to be used by each grid cell to find out its first and last corresponding index in the array *birth_particle_array*, as well as the individual number of new particles $\nu^{(c)}_{\mathrm{b},k+1}$ assigned to grid cell c in this time step.

Each grid cell splits this number into the number $\nu^{(c)}_{\mathrm{A},k+1}$ of new particles which are associated to the spatial measurement $g^{(c)}_{k+1}(z_{k+1}|x_{k+1})$ and the number $\nu^{(c)}_{\overline{\mathrm{A}},k+1}$ of new particles which are not associated. The relation is given by

$$\frac{\nu^{(c)}_{\mathrm{A},k+1}}{\nu^{(c)}_{\overline{\mathrm{A}},k+1}} = \frac{p^{(c)}_{\mathrm{A},k+1}}{1 - p^{(c)}_{\mathrm{A},k+1}}, \tag{79}$$

and

$$\nu^{(c)}_{\mathrm{A},k+1} + \nu^{(c)}_{\overline{\mathrm{A}},k+1} = \nu^{(c)}_{\mathrm{b},k+1}. \tag{80}$$

Each grid cell also calculates and stores the weights for associated (75) and unassociated (77) new particles. As a next step, each grid cell iterates over all its assigned new particles and defines it as an associated or unassociated particle, respectively, and sets the particle grid cell index. Finally, each particle initializes itself with a random initial state within its grid cell. Again, enough random numbers should be sampled in a separate step in advance so the initialization step just needs to look them up. This time, each particle uses the grid map as a lookup table for its initial position and weight.

6) Statistical Moments of Grid Cells: Algorithm 6 calculates the first two statistical moments of the two-dimensional velocity $[v_{\mathrm{x}}\, v_{\mathrm{y}}]^{\mathrm{T}}$ in a grid cell, considering all updated, normalized, persistent particles. If grid cell c contains a certain number of particles, the mean velocity component in x-direction $\overline{v}^{(c)}_{\mathrm{x}}$ can be approximated by

$$\overline{v}^{(c)}_{\mathrm{x}} \approx \frac{1}{\varrho^{(c)}_{\mathrm{p},k+1}} \sum_{i=1}^{\nu^{(c)}_{\mathrm{p},k+1}} w^{(i,c)}_{\mathrm{p},k+1} \cdot v^{(i,c)}_{\mathrm{x,p},k+1}, \tag{81}$$

and analogously for the component in y-direction. The symbol $v^{(i,c)}_{\mathrm{x,p},k+1}$ denotes the velocity x-component of a posterior persistent particle $x^{(i,c)}_{\mathrm{p},k+1}$ in grid cell c.

Recall that $\varrho^{(c)}_{\mathrm{p},k+1}$ is the persistent part of the posterior occupancy mass, which equals the sum of updated, normalized

weights $w_{\mathrm{p},k+1}^{(i,c)}$ of persistent particles in grid cell c:

$$\varrho_{\mathrm{p},k+1}^{(c)} = \sum_{i=1}^{\nu_{\mathrm{p},k+1}^{(c)}} w_{\mathrm{p},k+1}^{(i,c)}. \tag{82}$$

The variance of the velocity component in x-direction can be approximated by

$$\sigma^{2(c)}_{v_\mathrm{x}} \approx \frac{1}{\varrho_{\mathrm{p},k+1}^{(c)}} \sum_{i=1}^{\nu_{\mathrm{p},k+1}^{(c)}} w_{\mathrm{p},k+1}^{(i,c)} \cdot \left(v_{\mathrm{x,p},k+1}^{(i,c)}\right)^2 - \left(\overline{v}_\mathrm{x}^{(c)}\right)^2, \tag{83}$$

and analogously for the component in y-direction. The covariance of the velocity components in x- and y-direction can by approximated by

$$\sigma^{(c)}_{v_\mathrm{x}v_\mathrm{y}} \approx \frac{1}{\varrho_{\mathrm{p},k+1}^{(c)}} \sum_{i=1}^{\nu_{\mathrm{p},k+1}^{(c)}} w_{\mathrm{p},k+1}^{(i,c)} \cdot v_{\mathrm{x,p},k+1}^{(i,c)} \cdot v_{\mathrm{y,p},k+1}^{(i,c)} - \overline{v}_\mathrm{x}^{(c)} \cdot \overline{v}_\mathrm{y}^{(c)}. \tag{84}$$

Since particles are sorted by their grid cell index, the calculation of all sums in equations (81), (82), (83), and (84) can be realized by parallel accumulation of the according values. A parallel **for** loop over all grid cells then only subtracts the corresponding accumulated values. Again, the computational complexity is constant for each cell, independent of the individual number of particles in the cell at time step $k+1$. This is optimal with respect to load balancing between threads.

7) Resampling: Particles are resampled according to Algorithm 7 to avoid degeneration. The resampling step accumulates the normalized weights of persistent and new-born particles in the array *joint_weight_array_accum*. It draws ν sorted random numbers which are equally distributed between 0 and the sum of all particles and stores the random numbers in the array *rand_array*. Each random number falls into a certain interval of accumulated weights, which corresponds to a certain particle index. For each random number in *rand_array*, the corresponding particle is chosen and copied into the particle array for the next time step *particle_array_next*.

VIII. Evaluation

This section evaluates the Dempster-Shafer approximation of the PHD/MIB filter (DS-PHD/MIB filter) with real-world sensor data. The goal is to investigate if the DS-PHD/MIB filter performs as expected in different scenarios. A focus lies on the effect of the birth probability, which will be varied in all experiments.

A test vehicle equipped with laser and radar sensors is used for recording measurement data. In a first experiment, an object approaches the ego vehicle with varying speed. The evaluation examines the speed estimation performance and consistency of the DS-PHD/MIB filter. In a second experiment, the ego vehicle follows a dynamic object and the evaluation investigates the ability of the DS-PHD/MIB filter to separate dynamic and static obstacles in the vehicle's environment. The evaluation also determines the effect of fusing radar and laser data in comparison to using laser data only. Finally, the computation time of the parallel implementation with varying numbers of particles is analyzed.

TABLE I
Overview of the experimental parameters of the DS-PHD/MIB filter. Here, SD is an abbreviation for standard deviation.

Parameter	Symbol	Value
Grid map edge size	-	120 m
Grid cell edge size	-	0.1 m
Number of consistent particles	ν	$2 \cdot 10^6$
Number of new-born particles per step	ν_b	$2 \cdot 10^5$
Persistence probability	p_S	0.99
SD velocity new-born particles	$\sigma_{\mathrm{B,v}}$	4 m/s
SD process noise position	σ_p	$0.02\frac{\mathrm{m}}{T/\mathrm{s}}$
SD process noise velocity	σ_v	$0.8\frac{\mathrm{m/s}}{T/\mathrm{s}}$
Birth probability	p_B	0.005 ... 0.1

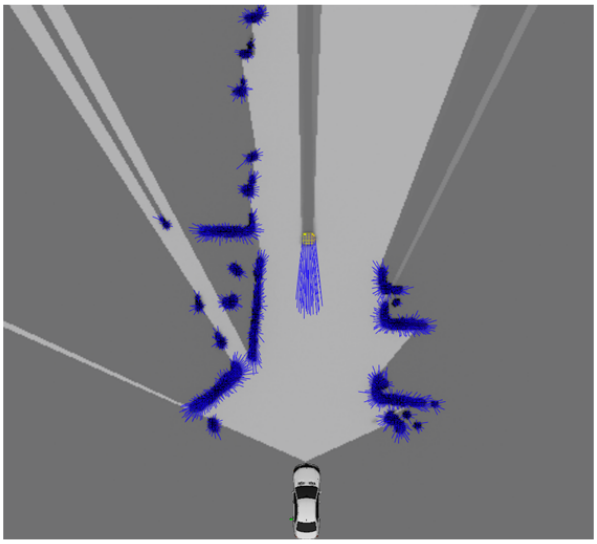


Fig. 8. Velocity estimation test scenario: A Segway approaches the test vehicle. The estimated mean velocity of every grid cell is visualized as a blue vector.

A. Experiment Configuration

The test vehicle is equipped with a Valeo four-layer laser scanner with an opening angle of 120 degrees in the front bumper. Additionally, two short range Delphi single beam mono pulse radars facing to the front left and front right sides cover a similar area. The vehicle speed and yaw rate are available via CAN messages, so the ego movement of the test vehicle can be compensated in the grid map.

The grid map covers an area of 120 m by 120 m with the test vehicle in the center. Each grid cell measures 10 cm by 10 cm. Table I shows the parameter set of the DS-PHD/MIB filter. The parallel implementation is tested on an Nvidia GTX980 GPU, supported by a single core of an Intel i7 processor.

B. Velocity Estimation of a Moving Object

In this test scenario, a Segway approaches the test vehicle starting from a distance of ca. 50 m, see Fig. 8. The Segway

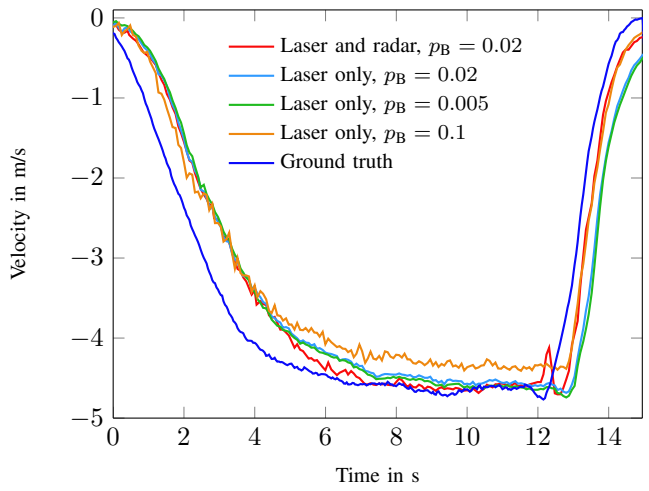


Fig. 9. Velocity estimation results under variation of the birth probability p_B. The Segway reaches the short range radar field of view after a time of ca. 5 s. Fusion with radar further improves the velocity estimation.

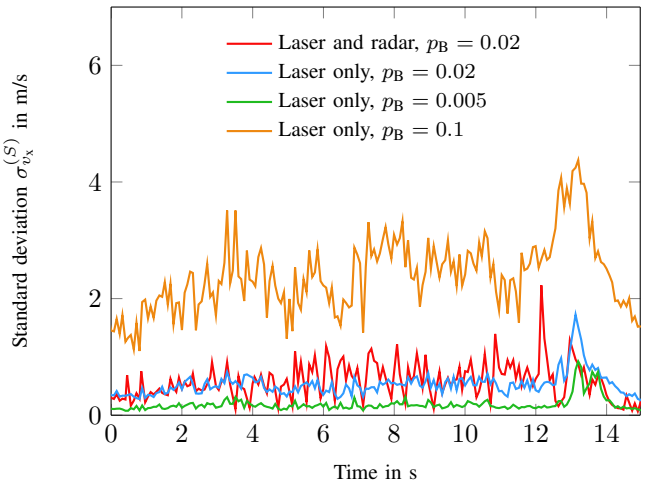


Fig. 10. Estimation standard deviation $\sigma_{v_x}^{(S)}$ of the Segway's velocity x-component provided by the DS-PHD/MIB filter under variation of the birth probability p_B.

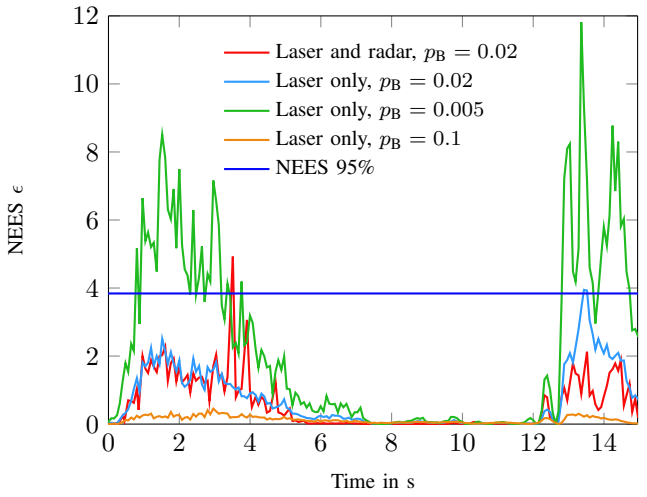


Fig. 11. Normalized estimation error squared (NEES) ϵ of the estimated velocity x-component under variation of the birth probability p_B.

accelerates slowly, drives with almost constant speed and then decelerates strongly. The set of grid cells representing the Segway is denoted S. The experiment evaluates the mean velocity $\overline{v}_x^{(S)}$ of these grid cells:

$$\overline{v}_x^{(S)} = \sum_{c \in S} \frac{1}{|S|} \overline{v}_x^{(c)}. \tag{85}$$

The x-component of the estimated velocity is aligned to the longitudinal axis of the test vehicle.

Figure 9 shows the results of the experiment. In the beginning the Segway accelerates slowly. Since the process model assumes constant velocity, the estimated velocity is delayed during the acceleration phase. During the constant velocity phase, the estimation converges closely to the real velocity for birth probabilities $p_B = 0.005$ and $p_B = 0.02$. Choosing a process model with a higher birth probability of $p_B = 0.1$ results in a persistent bias of the estimated velocity. The reason is that the mean of the birth distribution is zero, so new-born particles generally distort the velocity estimation towards zero. The results show that choosing an appropriate birth probability is important for the velocity estimation performance of the filter.

Fusion of radar data which contains Doppler measurements further improves the velocity estimation. In this realization, the Segway reaches the radar field of view at a time of ca. 4 s. Before that point in time, the Segway is outside the radar range and only rarely detected by radar. Radar Doppler measurements reduce even small remaining bias effects and lead to a much faster convergence during the strong deceleration phase in the end. The small peak after 12 s is assumed to be caused by a movement of the person riding the Segway, commonly referred to as micro-Doppler.

C. Consistency of the DS-MIB/PHD filter

This section evaluates the consistency of the DS-MIB/PHD filter in dependency of the birth probability p_B, focusing on the x-component of the estimated velocity. The DS-MIB/PHD filter provides for each grid cell the estimation variance $\sigma^{2(c)}_{v_x}$. The experiment considers the combined distribution of the set S of all grid cells representing the Segway as a Gaussian mixture. Hence the combined variance $\sigma^{2(S)}_{v_x}$ of the Segway is given by

$$\sigma^{2(S)}_{v_x} = \sum_{c \in S} \frac{1}{|S|} \left(\sigma^{2(c)}_{v_x} + \overline{v}^{2(c)}_x \right) - \overline{v}^{2(S)}_x. \tag{86}$$

This corresponds to the variance of all particles representing the Segway.

Figure 10 shows the standard deviation $\sigma_{v_x}^{(S)}$ of the velocity x-component of the Segway provided by the DS-MIB/PHD filter. The test scenario is the same as used for the velocity estimation. The uncertainty increases with the birth probability p_B due to the high number of new-born particles in the dynamic object.

To evaluate the consistency of the DS-PHD/MIB filter, the experiment calculates the normalized estimation error squared



Fig. 12. Test scenario for separation of static and dynamic grid cells. The color code represents the direction of movement, the color saturation is determined by the Mahalanobis distance between the velocity distribution and the velocity $v = 0$ in a grid cell.

(NEES) ϵ, given by [44]:

$$\epsilon = \frac{\left(\overline{v}_{\mathrm{x}}^{(S)} - v_{\mathrm{x}}^{(S)}\right)^2}{\sigma^{2\,(S)}_{v_{\mathrm{x}}}}, \tag{87}$$

where $v_{\mathrm{x}}^{(S)}$ is the true velocity x-component of the Segway.

Figure 11 shows the result of the consistency test and compares the NEES ϵ to the 95% level [44]. The applied process model (14) assumes constant velocity and does not model acceleration maneuvers. To compensate this, the filter designer can choose a higher velocity process noise than expected during constant velocity maneuvers as a trade-off between both modes. The consistency check shows why the birth probability p_{B} should not be chosen too small. Especially during acceleration maneuvers, the result can become inconsistent, because the filter underestimates the uncertainty of the estimation result, which happens during the deceleration phase with a birth probability of $p_{\mathrm{B}} = 0.005$.

D. Separation of Moving and Stationary Obstacles

In the second test scenario, the Segway drives along between parked vehicles with the ego vehicle following behind. Except for the Segway, the environment is static. In a manual post-processing step, all grid cells were labeled as dynamic or static. An example situation of the test is depicted in Fig. 12.

The evaluation uses the DS-PHD/MIB filter as a classifier to separate grid cells into dynamic or static. The criterion for the assignment is the Mahalanobis distance m between the estimated two-dimensional probability density $p(v) : \mathbb{R}^2 \to \mathbb{R}$ of the velocity distribution in a cell and the velocity $v = 0$. The density is approximated from the particle representation as a Gaussian distribution with mean $\overline{v}$ and covariance matrix P as

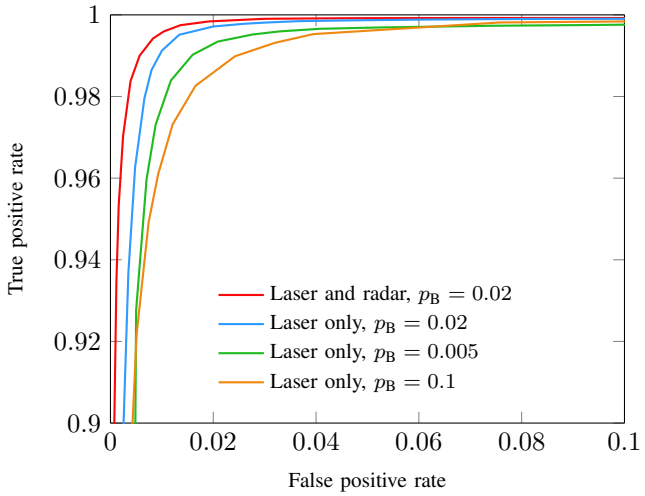


Fig. 13. Detection of dynamic grid cells with the DS-PHD/MIB filter under variation of the birth probability p_{B}.

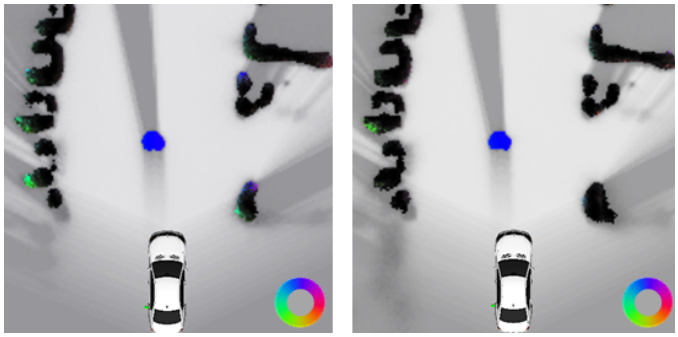


Fig. 14. Left: Laser only, false positive movements occur on both sides. Right: Fusion of laser and radar, false positive movements are reduced. The color code represents the direction of movement.

calculated in (81), (83), (84). Then the Mahalanobis distance is given by

$$m = \overline{v} P^{-1} \overline{v}^T. \tag{88}$$

Applying different threshold values τ_m, the evaluation classifies grid cells as a static detection if $m < \tau_m$ or as a dynamic detection if $m \geq \tau_m$.

Figure 13 shows the receiver operating characteristic (ROC) curve of the assignment. It shows that the DS-PHD/MIB filter is able to achieve a true positive rate of 99% (ratio of correctly detected dynamic cells to total number of dynamic cells) at a false positive rate of 1% (ratio of falsely detected dynamic cells to total number of static cells) in the test scenario. The birth probability p_{B} of the process model has an important influence on the estimation result. Best classification performance is achieved with a birth probability value of $p_{\mathrm{B}} = 0.02$, which also delivered consistent estimation results in the previous experiment.

Again, fusion with radar data further improves the overall result. As described in details in [20], Doppler measurements help reduce false positive movement estimation in grid cells. An exemplary visualization of the dynamic grid map with and without radar is given in Fig. 14.

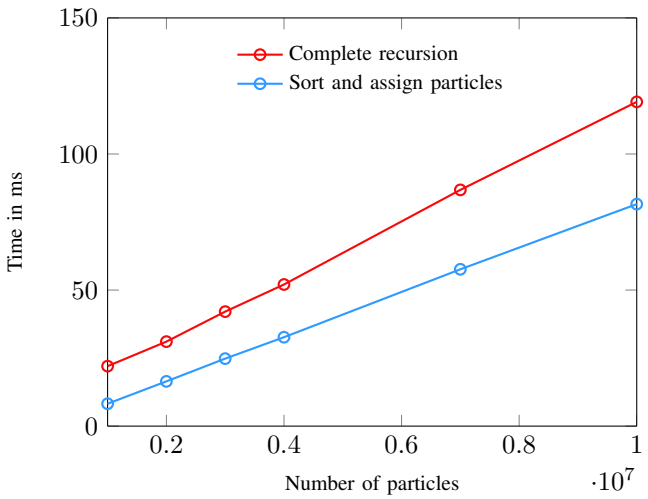


Fig. 15. Parallel implementation of the DS-PHD/MIB filter: Computation time with varying number of particles and $1.44 \cdot 10^6$ grid cells.

E. Computation Time

The parallel implementation presented in Sect. V allows to run the DS-PHD/MIB filter in real-time applications. Figure 15 shows the computation time of one recursion of the DS-PHD/MIB filter in dependence on the number of persistent particles ν, running on an Nvidia GTX980 GPU. The number of new-born particles ν_b is chosen to be 10% of the number of persistent particles in this experiment.

As discussed in Sect. V, the time complexity of the sorting step is above linear. However, in the range between $1 \cdot 10^6$ and $10 \cdot 10^6$ particles, the absolute computation time relates approximately linear to the number of particles. A typical environment perception application with a refresh time of ca. 50 ms can process more then $2 \cdot 10^6$ particles and $1.44 \cdot 10^6$ grid cells in each update step, which are also the numbers that have been used during the experiments.

IX. Conclusion and Outlook

This paper presented the first mathematically rigorous approach for the dynamic state estimation of grid cells for robotic or vehicle environment perception based on random finite sets (RFSs). The PHD/MIB filter approximates the multi-object estimation problem by combining the probability hypothesis density filter and multiple instances of the Bernoulli filter. In contrast to former approaches, the top-down derivation of the proposed PHD/MIB filter facilitates a characterization of the approximation error. Further, the proposed RFS formulation provides an explicit, stochastic birth model for appearing objects as well as a physical meaning for the densities represented by the particles. The validity of the PHD/MIB filter was additionally verified by the proof that the filter corresponds to the well-known binary Bayes filter in case of a static process model. Moreover, an approximate particle realization of the PHD/MIB filter in the Dempster-Shafer domain called DS-PHD/MIB filter was proposed which facilitates a real-time capable implementation for practical applications in robotics or vehicle environment perception since it requires a significantly smaller number of particles. Further, an efficient parallel algorithm suitable for a GPU implementation of the filter was presented as pseudo code.

The quantitative evaluation with real-world sensor data showed that appropriate stochastic models for the system process and for the observation process lead to consistent estimation results. The experiments have confirmed that the DS-PHD/MIB filter provides useful results in regard to velocity estimation of dynamic obstacles and separation of dynamic and static obstacles.

Further research should investigate possibilities of explicitly modeling the dynamic behavior of free space, which is bound to a naive model in the DS-PHD/MIB approximation. Theoretically, the PHD/MIB filter is able to model possible movements in occluded areas which could lead to useful applications.

References

[1] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.

[2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. Cambridge: The MIT Press, 2005.

[3] K. Dietmayer, S. Reuter, and D. Nuss, "Representation of Fused Environment Data," in *Handbook of Driver Assistance Systems*, H. Winner, S. Hakuli, F. Lotz, and C. Singer, Eds. Cham: Springer International Publishing, 2015.

[4] D. Nuss, B. Wilking, J. Wiest, H. Deusch, S. Reuter, and K. Dietmayer, "Decision-Free True Positive Estimation with Grid Maps for Multi-Object Tracking," in *Proceedings of the IEEE Conference on Intelligent Transportation Systems*, 2013, pp. 28–34.

[5] C. Urmson *et al.*, "Autonomous Driving in Urban Environments: Boss and the Urban Challenge," *Journal of Field Robotics, Special Issue on the 2007 DARPA Urban Challenge, Part I*, vol. 25, no. 8, pp. 425–466, 2008.

[6] F. Kunz, D. Nuss, J. Wiest, H. Deusch, S. Reuter, F. Gritschneder, A. Scheel, M. Stübler, M. Bach, P. Hatzelmann, C. Wild, and K. Dietmayer, "Autonomous Driving at Ulm University: A Modular, Robust, and Sensor-Independent Fusion Approach," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2015, pp. 666–673.

[7] C. Laugier, I. Paromtchik, M. Perrollaz, M. Yong, J.-D. Yoder, C. Tay, K. Mekhnacha, and A. Nègre, "Probabilistic Analysis of Dynamic Scenes and Collision Risks Assessment to Improve Driving Safety," *IEEE Intelligent Transportation Systems Magazine*, vol. 3, no. 4, pp. 4–19, 2011.

[8] D. Nuss, M. Thom, A. Danzer, and K. Dietmayer, "Fusion of Laser and Monocular Camera Data in Object Grid Maps for Vehicle Environment Perception," in *Proceedings of the International Conference on Information Fusion*, 2014.

[9] C.-C. Wang, C. Thorpe, S. Thrun, M. Hebert, and H. Durrant-Whyte, "Simultaneous Localization, Mapping and Moving Object Tracking," *International Journal of Robotics Research*, vol. 26, no. 9, pp. 889–916, 2007.

[10] M. Bouzouraa and U. Hofmann, "Fusion of Occupancy Grid Mapping and Model Based Object Tracking for Driver Assistance Systems using Laser and Radar Sensors," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2010, pp. 294–300.

[11] T.-D. Vu and O. Aycard, "Laser-Based Detection and Tracking Moving Objects using Data-Driven Markov Chain Monte Carlo," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2009, pp. 3800–3806.

[12] D. Nuss, M. Stübler, and K. Dietmayer, "Consistent Environmental Modeling by use of Occupancy Grid Maps, Digital Road Maps, and Multi-Object Tracking," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2014, pp. 1371–1377.

[13] A. Vatavu, R. Danescu, and S. Nedevschi, "Stereovision-Based Multiple Object Tracking in Traffic Scenarios using Free-Form Obstacle Delimiters and Particle Filters," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 1, pp. 498–511, 2015.

[14] C. Coué, C. Pradalier, C. Laugier, T. Fraichard, and P. Bessière, "Bayesian Occupancy Filtering for Multitarget Tracking: An Automotive Application," *International Journal of Robotics Research*, vol. 25, no. 1, pp. 19–30, 2006.

[15] T. Gindele, S. Brechtel, J. Schröder, and R. Dillmann, "Bayesian Occupancy Grid Filter for Dynamic Environments using Prior Map Knowledge," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2009, pp. 669–676.

[16] R. Danescu, F. Oniga, and S. Nedevschi, "Particle Grid Tracking System for Stereovision Based Environment Perception," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2010, pp. 987–992.

[17] R. Danescu, F. Oniga, and S. Nedevschi, "Modeling and Tracking the Driving Environment with a Particle-Based Occupancy Grid," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 4, pp. 1331–1342, 2011.

[18] A. Nègre, L. Rummelhard, and C. Laugier, "Hybrid Sampling Bayesian Occupancy Filter," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2014, pp. 1307–1312.

[19] G. Tanzmeister, J. Thomas, D. Wollherr, and M. Buss, "Grid-Based Mapping and Tracking in Dynamic Environments using a Uniform Evidential Environment Representation," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2014, pp. 6090–6095.

[20] D. Nuss, T. Yuan, G. Krehl, M. Stübler, S. Reuter, and K. Dietmayer, "Fusion of Laser and Radar Sensor Data with a Sequential Monte Carlo Bayesian Occupancy Filter," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2015, pp. 1074–1081.

[21] R. Mahler, *Statistical Multisource-Multitarget Information Fusion*. Norwood: Artech House, 2007.

[22] R. Mahler, "Multitarget Bayes Filtering via First-Order Multitarget Moments," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 39, no. 4, pp. 1152–1178, 2003.

[23] B. Ristic, *Particle Filters for Random Set Models*. New York: Springer, 2013.

[24] A. Dempster, "A Generalization of Bayesian Inference," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 30, no. 2, pp. 205–247, 1968.

[25] R. Kálmán, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[26] B.-N. Vo, S. Singh, and A. Doucet, "Sequential Monte Carlo Methods for Multitarget Filtering with Random Finite Sets," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 41, no. 4, pp. 1224–1245, 2005.

[27] S. Reuter, B. Wilking, J. Wiest, M. Munz, and K. Dietmayer, "Real-Time Multi-Object Tracking using Random Finite Sets," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 49, no. 4, pp. 2666–2678, 2013.

[28] B.-T. Vo and B.-N. Vo, "Labeled Random Finite Sets and Multi-Object Conjugate Priors," *IEEE Transactions on Signal Processing*, vol. 61, no. 13, pp. 3460–3475, 2013.

[29] B.-N. Vo, B.-T. Vo, and D. Phung, "Labeled Random Finite Sets and the Bayes Multi-Target Tracking Filter," *IEEE Transactions on Signal Processing*, vol. 62, no. 24, pp. 6554–6567, 2014.

[30] B.-N. Vo and W.-K. Ma, "The Gaussian Mixture Probability Hypothesis Density Filter," *IEEE Transactions on Signal Processing*, vol. 54, no. 11, pp. 4091–4104, 2006.

[31] R. Mahler, "PHD Filters of Higher Order in Target Number," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 43, no. 4, pp. 1523–1543, 2007.

[32] B.-T. Vo, B.-N. Vo, and A. Cantoni, "Analytic Implementations of the Cardinalized Probability Hypothesis Density Filter," *IEEE Transactions on Signal Processing*, vol. 55, no. 7, pp. 3553–3567, 2007.

[33] B.-T. Vo, B.-N. Vo, and A. Cantoni, "The Cardinality Balanced Multi-Target Multi-Bernoulli Filter and Its Implementations," *IEEE Transactions on Signal Processing*, vol. 57, no. 2, pp. 409–423, 2009.

[34] S. Reuter, B.-T. Vo, B.-N. Vo, and K. Dietmayer, "The Labeled Multi-Bernoulli Filter," *IEEE Transactions on Signal Processing*, vol. 62, no. 12, pp. 3246–3260, 2014.

[35] G. Shafer, *A Mathematical Theory of Evidence*. Princeton: Princeton University Press, 1976.

[36] P. Smets, "The Combination of Evidence in the Transferable Belief Model," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 12, no. 5, pp. 447–458, 1990.

[37] M. Konrad and K. Dietmayer, "Occupancy Grid Mapping using the Dempster-Shafer-Theory," in *Proceedings of the International Workshop on Intelligent Transportation (WIT)*, 2011, pp. 167–172.

[38] J. Moras, V. Cherfaoui, and P. Bonnifait, "Moving Objects Detection by Conflict Analysis in Evidential Grids," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2011, pp. 1122–1127.

[39] N. Satish, M. Harris, and M. Garland, "Designing Efficient Sorting Algorithms for Manycore GPUs," in *Proceedings of the IEEE International Symposium on Parallel & Distributed Processing*, 2009.

[40] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2001, pp. 511–518.

[41] NVIDIA, *CUDA CURAND Library*, NVIDIA Corporation, Santa Clara, CA, USA, 2016. [Online]. Available: http://docs.nvidia.com/cuda/curand

[42] J. Hoberock and N. Bell, "Thrust: A parallel template library," 2016, version 1.8.1. [Online]. Available: http://thrust.github.io/

[43] F. Homm, N. Kämpchen, J. Ota, and D. Burschka, "Efficient Occupancy Grid Computation on the GPU with Lidar and Radar for Road Boundary Detection," in *Proceedings of the IEEE Intelligent Vehicles Symposium*, 2010, pp. 1006–1013.

[44] Y. Bar-Shalom, X.-R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York: John Wiley & Sons, Inc., 2001.